

BLAZOR

SUCCINCTLY

BY **MICHAEL WASHINGTON**

Blazor Succinctly

Michael Washington

Foreword by Daniel Jebaraj



Copyright © 2026 by Syncfusion, Inc.

2501 Aerial Center Parkway

Suite 111

Morrisville, NC 27560

USA

All rights reserved.

ISBN: 978-1-64200-252-2

Important licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal or educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

SYNCFUSION, SUCCINCTLY, DELIVER INNOVATION WITH EASE, ESSENTIAL, and .NET ESSENTIALS are the registered trademarks of Syncfusion, Inc.

Technical Reviewer: James McCaffrey

Copy Editor: Naomi Spicer, content producer, Syncfusion, Inc.

Acquisitions Coordinator: Graham High, content team lead, Syncfusion, Inc.

Proofreader: Graham High, content team lead, Syncfusion, Inc.

Table of Contents

The <i>Succinctly</i> Series of Books	7
About the Author	8
Introduction.....	9
Chapter 1 What is Blazor?	10
Server-side Blazor	10
Client-side Blazor (WebAssembly).....	11
Blazor Hybrid	12
Render modes in Blazor	13
Blazor Server	13
Blazor WebAssembly.....	14
Blazor Hybrid	14
Core Blazor features	14
Components and routing.....	14
Parameters	16
Data binding	17
Events	18
Chapter 2 The Help Desk Application.....	20
Help desk users.....	20
Help desk administrators	26
Chapter 3 Create the Help Desk Application.....	29
Install SQL Server	30
Install .NET Core and Visual Studio.....	30
Create the project.....	31
Create the database.....	33

Enable role management	35
Chapter 4 Explore the Project	39
Startup	39
Routing.....	41
Layouts	41
Chapter 5 Add Syncfusion	43
Install NuGet packages	43
Additional configuration	44
Chapter 6 Creating a Data Layer	46
Create the database tables	46
Create the DataContext by using EF Core tools	49
Set the database connection	53
Create the SyncfusionHelpDeskService	54
Register the SyncfusionHelpDeskService.....	57
Create the administrator	58
Chapter 7 Creating New Tickets.....	64
Blazor Toast component.....	64
Forms and validation	64
Forms	64
Validation.....	65
HelpDeskTicket class.....	65
Syncfusion Blazor controls.....	66
New ticket form.....	67
Test the form	69
Chapter 8 Help Desk Ticket Administration	73
Create the ticket administration page	73

Add link in NavMenu.razor	74
Using the Syncfusion DataGrid.....	76
Deleting a record	80
Syncfusion Dialog control	80
Using @ref to capture references to components.....	81
Edit ticket control	83
Chapter 9 Sending Emails	90
Email using SendGrid.....	90
Email sender class	91
Send emails: New help desk ticket	94
Send emails: Updated help desk ticket.....	94
Route parameters.....	95
Email link.....	98
Chapter 10 Creating Tickets Using AI.....	100
Set up the Smart Paste Button component.....	100
Using the Smart Paste Button	102
Conclusion	104

The *Succinctly* Series of Books

Daniel Jebaraj
CEO of Syncfusion, Inc.

When we published our first *Succinctly* series book in 2012, *jQuery Succinctly*, our goal was to produce a series of concise technical books targeted at software developers working primarily on the Microsoft platform. We firmly believed then, as we do now, that most topics of interest can be translated into books that are about 100 pages in length.

We have since published over 200 books that have been downloaded millions of times. Reaching more than 2.7 million readers around the world, we have more than 70 authors who now cover a wider range of topics, such as Blazor, machine learning, and big data.

Each author is carefully chosen from a pool of talented experts who share our vision. The book before you and the others in this series are the result of our authors' tireless work. Within these pages, you will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

We are absolutely thrilled with the enthusiastic reception of our books. We believe the *Succinctly* series is the largest library of free technical books being actively published today. Truly exciting!

Our goal is to keep the information free and easily available so that anyone with a computing device and internet access can obtain concise information and benefit from it. The books will always be free. Any updates we publish will also be free.

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctlyseries@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Please follow us on social media and help us spread the word about the *Succinctly* series!



About the Author

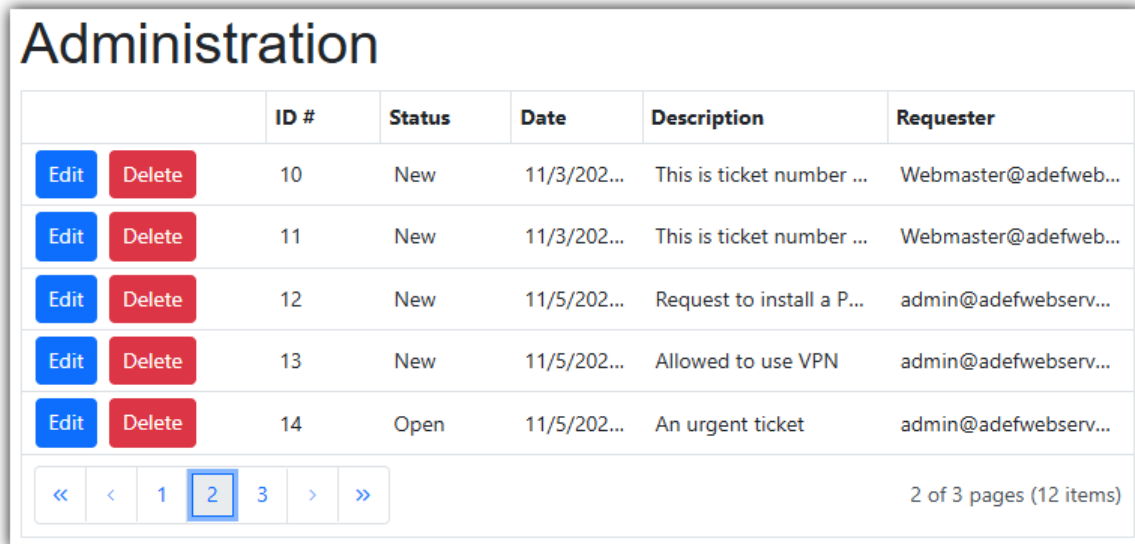
Michael Washington is a Microsoft MVP, an ASP.NET C# programmer, and the founder of BlazorHelpWebsite.com, BlazorData.net, and AIStoryBuilders.com. He is the author of *An Introduction to Building Applications with Blazor*. He has extensive knowledge in process improvement, billing systems, and student information systems. He has a son, Zachary, and resides in Los Angeles with his wife, Valerie.

You can follow him on BlueSky at bsky.app/profile/adeftwebserver.com.

Introduction

Blazor is an exciting technology that allows you to create web-based applications using C# instead of JavaScript. However, you can still implement custom JavaScript when you desire.

Blazor is an alternative to other single-page application frameworks such as Angular, React, or Vue.



	ID #	Status	Date	Description	Requester
Edit Delete	10	New	11/3/202...	This is ticket number ...	Webmaster@adefweb...
Edit Delete	11	New	11/3/202...	This is ticket number ...	Webmaster@adefweb...
Edit Delete	12	New	11/5/202...	Request to install a P...	admin@adefwebserv...
Edit Delete	13	New	11/5/202...	Allowed to use VPN	admin@adefwebserv...
Edit Delete	14	Open	11/5/202...	An urgent ticket	admin@adefwebserv...

« < 1 **2** 3 > » 2 of 3 pages (12 items)

Figure 1: Help desk administration

In this ebook, we will cover the core elements of Blazor and then explore additional features by building a sample application called Syncfusion Help Desk.

This application will demonstrate:

- Implementing authentication and authorization.
- Inserting, updating, and deleting data from the database.
- Using forms and validation.
- Implementing email notifications.
- Implementing AI features such as smart paste.

The code for this book is available on [GitHub](#). The step-by-step instructions use the Visual Studio 2022 Community edition. You can download Visual Studio Community for free at [this link](#). You can also create Blazor applications using the Visual Studio Code IDE. See dotnet.microsoft.com/en-us/learn/aspnet/blazor-tutorial/intro for more information. In addition, the SQL Server Developer edition is recommended, and a free download is available at [this link](#).

You may want to bookmark the official Microsoft Blazor documentation available [here](#).

Chapter 1 What is Blazor?

Blazor applications are composed of components that are constructed using C#, HTML-based Razor syntax, and CSS.

Blazor has three different hosting models: server-side Blazor, client-side Blazor (also known as Blazor WebAssembly), and Blazor Hybrid. Server-side Blazor and client-side Blazor run in all modern web browsers, including web browsers on mobile phones. Blazor Hybrid applications run natively in .NET on platforms such as Windows, macOS, and Android.

Server-side Blazor

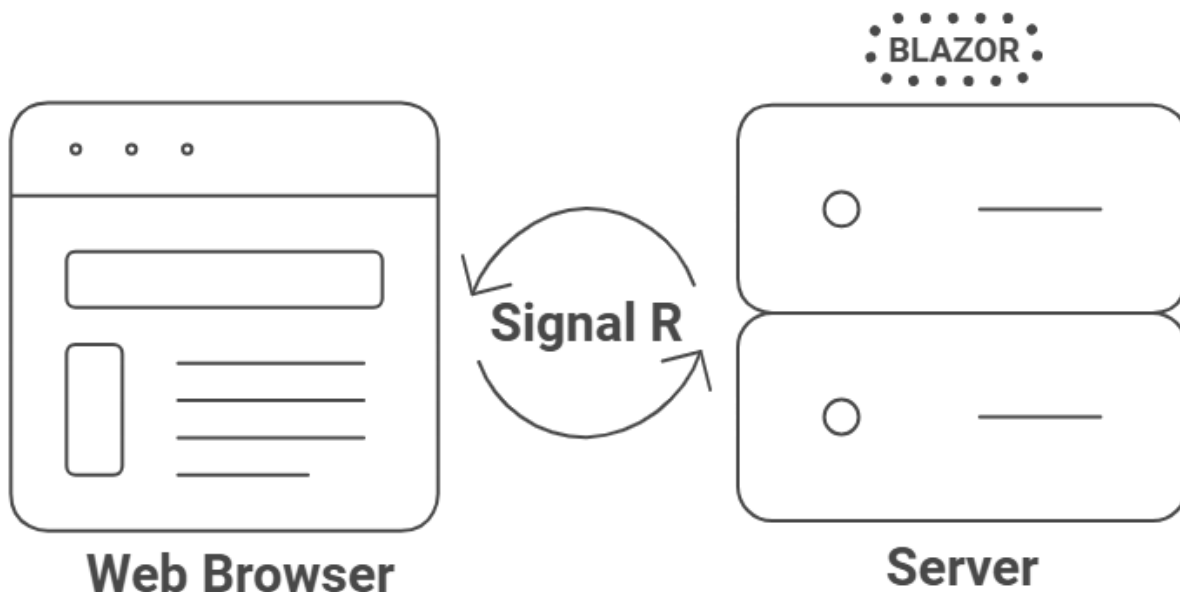


Figure 2: Server-side Blazor

Server-side Blazor renders the Razor components on the server and updates the webpage using a SignalR connection. The Blazor framework sends events from the web browser, such as button clicks and mouse movements, to the server. The Blazor runtime computes changes to the components on the server and sends a *diff-based* webpage back to the web browser.

The Syncfusion Help Desk sample application covered in this book will be built using server-side Blazor.

Client-side Blazor (WebAssembly)



Figure 3: Client-side Blazor (WebAssembly)

Client-side Blazor is composed of the same code as server-side Blazor, but it runs entirely in the web browser using a technology known as WebAssembly.

The primary difference between applications that are created in server-side Blazor versus client-side Blazor is that the client-side Blazor applications need to make web calls to access server data, whereas the server-side Blazor applications can omit this step since all their code is executed on the server.

One way to think of Blazor is as a framework for creating single-page application webpages using one of two architectures—client-side or server-side—using Razor technology written with the C# language.

Blazor Hybrid

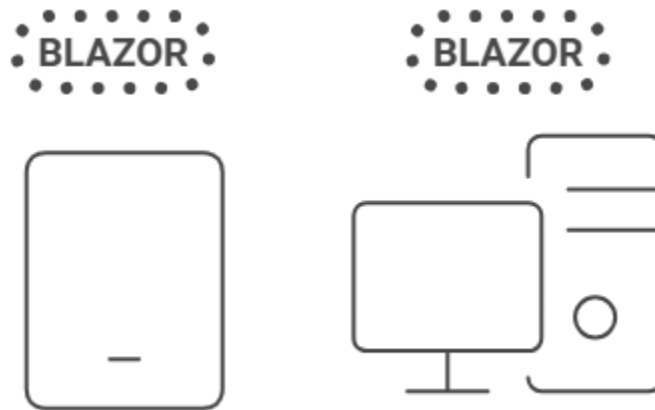


Figure 4: Blazor Hybrid

Blazor Hybrid applications, also known as .NET MAUI Blazor apps, run natively in .NET on platforms such as Windows, macOS, and Android. Unlike server-side Blazor and client-side Blazor, which run in web browsers, Blazor Hybrid leverages the native capabilities of the host platform to provide enhanced performance and functionality. This model allows developers to create applications that can run across multiple platforms using a single code base, providing a seamless and integrated experience for users.



Note: *Blazor Hybrid apps can also be built with WPF and Windows Forms, but for the purposes of this book, a Blazor Hybrid app will always refer to a .NET MAUI Blazor app.*

With Blazor Hybrid, developers can utilize platform-specific features and APIs, delivering a rich and responsive user experience. The Blazor framework handles the rendering of components and updates in real time, ensuring that applications remain interactive and efficient.

Render modes in Blazor

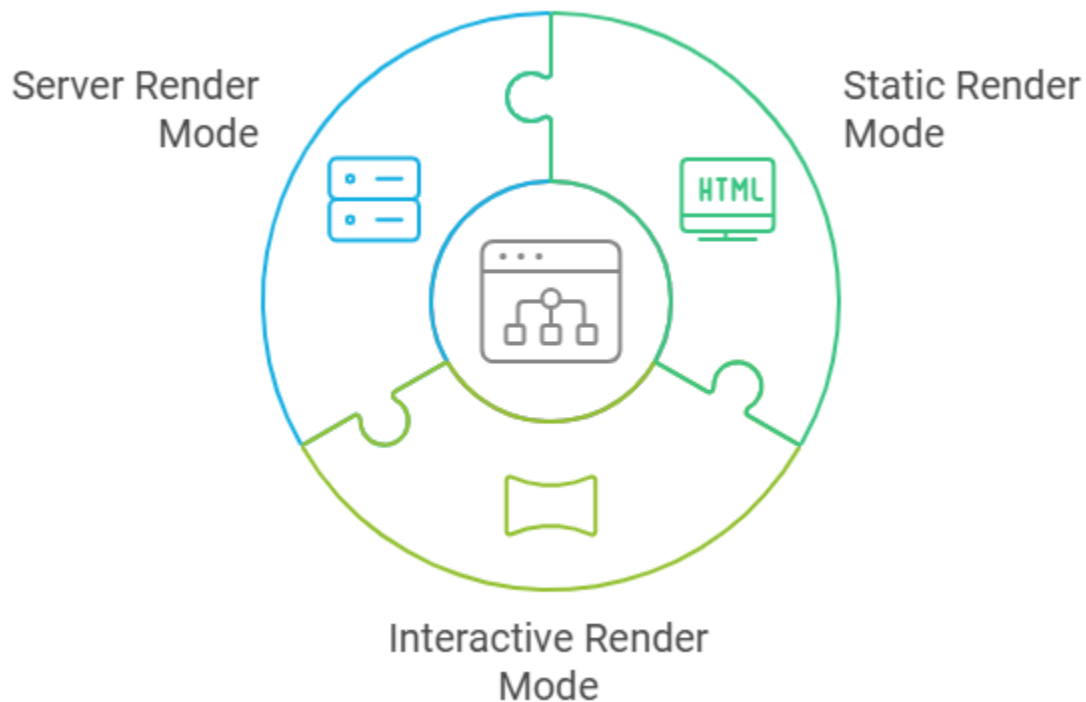


Figure 5: Blazor render modes

Blazor supports different **render modes** to optimize performance and user experience:

- **Static render mode:** Renders static HTML content without any interactivity, which is useful for content that doesn't require dynamic updates.
- **Interactive render mode:** Enables full interactivity by allowing the Blazor framework to handle events and update the UI dynamically.
- **Server render mode:** Executes components on the server and updates the client over a SignalR connection, similar to server-side Blazor.

The following is how each render mode is used with different Blazor hosting models:

Blazor Server

- **Static render mode:** Not typically used; Blazor Server is designed for dynamic interactivity via SignalR.
- **Interactive render mode:** Fully supported, allowing full interactivity through dynamic UI updates.
- **Server render mode:** Default mode for Blazor Server, with components running on the server and updating the client over SignalR.

Blazor WebAssembly

- **Static render mode:** Usable for static content but not the primary use case; Blazor WebAssembly focuses on client-side interactivity.
- **Interactive render mode:** Fully supported, enabling rich client-side interactivity by running .NET code in the browser.
- **Server render mode:** Not applicable; Blazor WebAssembly operates entirely on the client side.

Blazor Hybrid

- **Static render mode:** Less relevant; Blazor Hybrid combines WebAssembly and .NET MAUI for interactive applications.
- **Interactive render mode:** Fully supported, allowing interactive content using both WebAssembly and .NET MAUI.
- **Server render mode:** Partially relevant; integrates with server-side logic for certain operations, but not like Blazor Server.

Core Blazor features

Components and routing

A Blazor application is composed of components. A component is a chunk of code consisting of the user interface and the processing logic. A Blazor component is also called a Razor component.

Blazor features routing, where you can provide navigation to your controls using the `@page` directive followed by a unique route in quotation marks, preceded by a slash.

The following is an example of a simple Razor component called `ComponentExample.razor`:

Code Listing 1: ComponentExample.razor

```
@page "/componentexample"
<h3>This is Component Example</h3>
@code {
}
```

Figure 6 shows what the component looks like in a running application.



Figure 6: A simple component

A Razor component is contained in a Razor file and can be nested inside other components.

For example, we can create a component named `ComponentOne.razor` by using the code in Code Listing 2.

Code Listing 2: `ComponentOne.razor`

```
<h4 style="background-color:goldenrod">
    This is ComponentOne
</h4>

@code {
}
```

We can alter `ComponentExample.razor` to contain `ComponentOne.razor` as in Code Listing 3.

Code Listing 3: `ComponentExample.razor`

```
@page "/componentexample"

<h3>This is Component Example</h3>

<ComponentOne />

@code {
}
```

Figure 7 shows what `ComponentExample.razor` now looks like when running in the application.



Figure 7: A nested component



Note: A component's name must start with an uppercase character.

Parameters

Razor components can pass values to other components using parameters. Component parameters are defined using the `[Parameter]` attribute, which must be associated with a public variable.

For example, we can create a Razor component called `ParameterExampleComponent.razor` that contains a parameter called `Title` using the following code:

Code Listing 4: `ParameterExampleComponent.razor`

```
<h4>Parameter Example Component</h4>

<h5 style="color:red">@Title</h5>

@code {
    [Parameter]
    public string Title { get; set; }
}
```

We can create another Razor component called `ParameterExample.razor` that consumes the `ParameterExampleComponent.razor` control and passes a value (**Passed from Parent**) to the `Title` parameter in the `ParameterExampleComponent.razor` control.

Code Listing 5: `ParameterExample.razor`

```
@page "/parameterexample"

<h4>Parameter Example</h4>
```



```
<ParameterExampleComponent Title="Passed from Parent" />

@code {

}
```

When we run the application, we get the result shown in Figure 8.

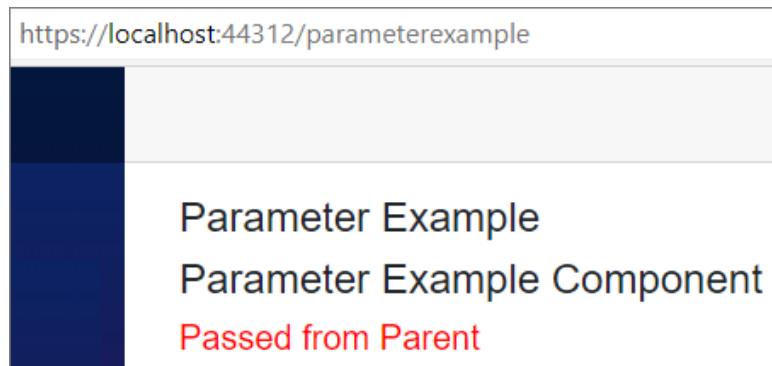


Figure 8: Parameter example

Data binding

Simple one-way binding in Blazor is achieved by declaring a parameter and referencing it using the @ symbol. An example of this is shown in Code Listing 6.

Code Listing 6: One-way binding

```
<b>BoundValue:</b> @BoundValue

@code {
    private string BoundValue { get; set; }

    protected override void OnInitialized()
    {
        BoundValue = "Initial Value";
    }
}
```

This displays the following when rendered:

BoundValue: Initial Value

Figure 9: One-way binding

Two-way dynamic databinding in Razor components is implemented using the **@bind** attribute. Code Listing 7 demonstrates this.

Code Listing 7: Two-way binding

```
<input @bind="BoundValue" @bind:event="oninput" />
<p>Display CurrentValue: @BoundValue</p>
@code {
    private string BoundValue { get; set; }
}
```

When we run the code, it displays the value entered in the text input box as it's typed.

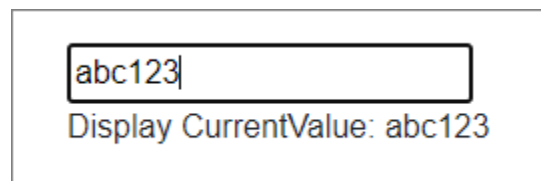


Figure 10: Two-way binding

Events

Raising events in Razor components is straightforward. Code Listing 8 demonstrates using the **@onclick** event handler to execute a method (**IncrementCount**) when the button is clicked.

Code Listing 8: A simple event

```
<p>Current count: @currentCount</p>
<button class="btn btn-primary" @onclick="IncrementCount">
    Click me
</button>
@code {
    private int currentCount = 0;

    private void IncrementCount()
    {
        currentCount++;
    }
}
```

When the control is rendered and the button is clicked six times, the UI looks like Figure 11.

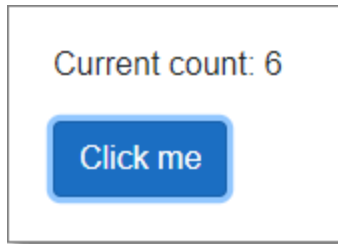
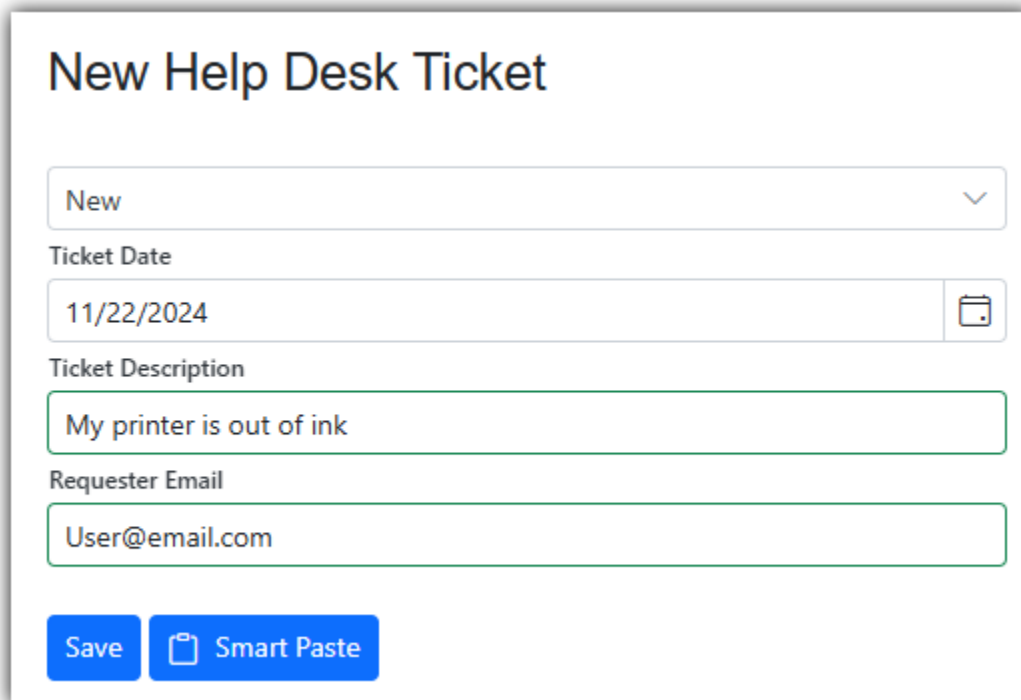


Figure 11: A simple event

Chapter 2 The Help Desk Application

To demonstrate the features of Blazor and how controls like those provided by Syncfusion can make development faster and easier, we will create a simple help desk application.

Help desk users



The screenshot shows a web form titled "New Help Desk Ticket". It contains the following fields and controls:

- A dropdown menu with the text "New" and a downward arrow.
- A "Ticket Date" label above a text input field containing "11/22/2024" and a calendar icon on the right.
- A "Ticket Description" label above a text input field containing "My printer is out of ink".
- A "Requester Email" label above a text input field containing "User@email.com".
- At the bottom, there are two blue buttons: "Save" and "Smart Paste" (which includes a clipboard icon).

Figure 12: New help desk ticket

When you run the application, users will see a form that allows them to create a new help desk ticket.

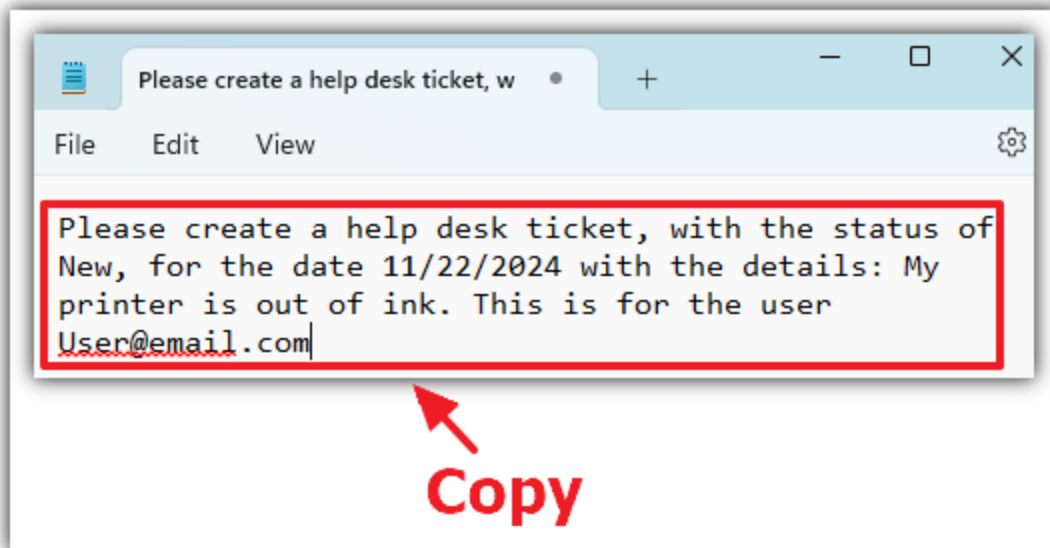


Figure 13: Copy help desk ticket contents

To create a help desk ticket, you can optionally copy information from an external source, such as an email or a text file that contains information for the help desk ticket.

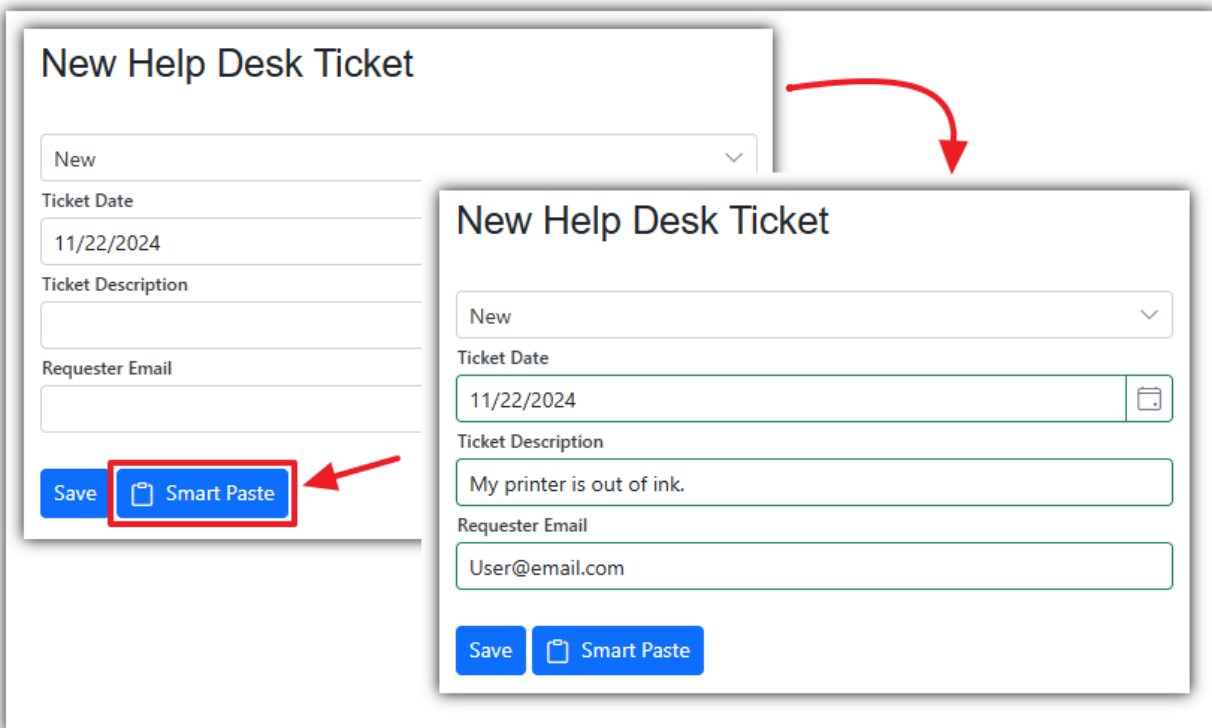


Figure 14: Use the Smart Paste button

You can then select Smart Paste to use AI to parse the copied information and automatically fill in the help desk ticket form.

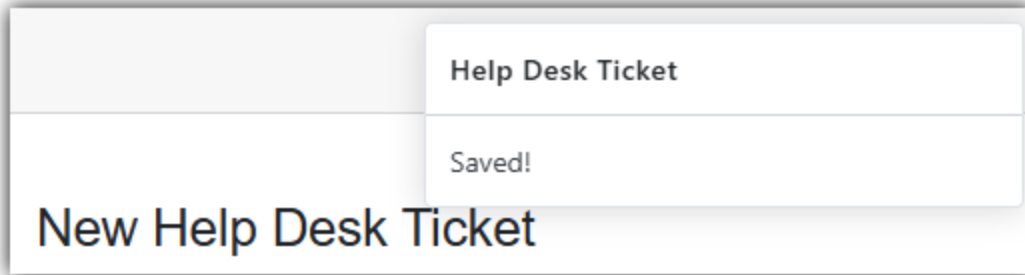


Figure 15: Syncfusion Toast

After you submit the help desk ticket by selecting the Save button, a pop-up window that uses the Syncfusion Toast control is displayed to let you know that your action was successful.

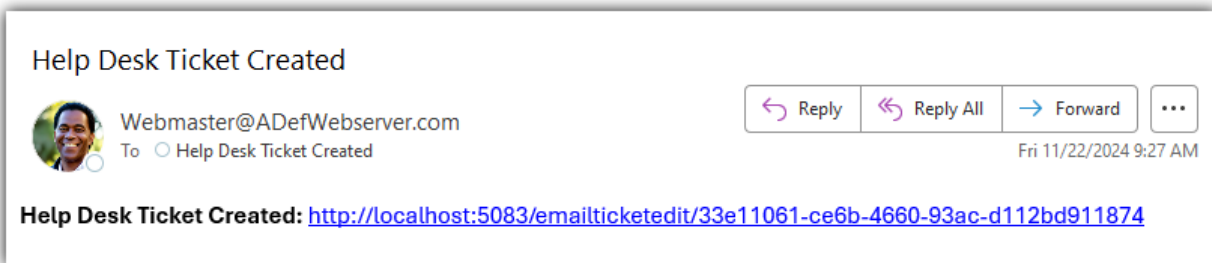


Figure 16: Email notification

An email is sent to the administrator with a link that will navigate directly to the help desk ticket.

EDIT TICKET # 3

Ticket Status
New

Ticket Date
11/22/2024

Ticket Description
My printer is out of ink.

Requester Email
User@email.com

Date	Description
No records to display	

What type of printer do you have?

Add Detail

Save

Figure 17: Administrator adding details

When the administrator is logged in and they click on the link in the email, they will have the ability to edit all fields and enter help desk ticket details at the bottom of the form by entering the desired text and clicking the Add Detail button.

EDIT TICKET # 3

×

Ticket Status

New

▼

Ticket Date

11/22/2024

📅

Ticket Description

My printer is out of ink.

Requester Email

User@email.com

Date	Description
11/28/2024	What type of printer do you have?

New Help Desk Ticket Detail

Add Detail

Save

Figure 18: Administrator saving details

After the desired help desk ticket details have been added, the administrator can select the Save button to save the record and send a notification email to the user who created the help desk ticket.

EDIT TICKET # 3

Disabled

Ticket Status

New

Ticket Date

11/22/2024

Ticket Description

My printer is out of ink.

Requester Email

User@email.com

Date	Description
11/28/2024	What type of printer do you have?

I have a HP101

1

Add Detail

2

3

Save

Figure 19: User adding details

The user who created the help desk ticket will receive an email with a link that navigates them to the help desk ticket and allows them to enter details. The fields at the top of the help desk ticket, however, are grayed out and disabled for them. They can only add new details.

Help desk administrators

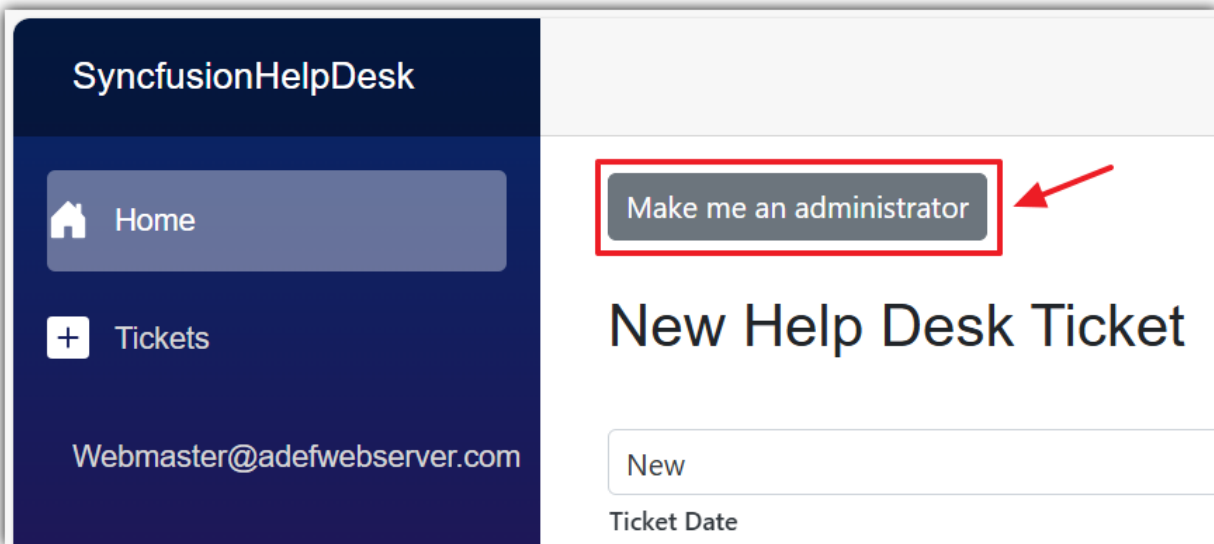


Figure 20: Administration menu link

A logged-in user who's not already an administrator will see a button to make themselves an administrator. Clicking this button will put the user in the administrator role and allow them to administer all help desk tickets.

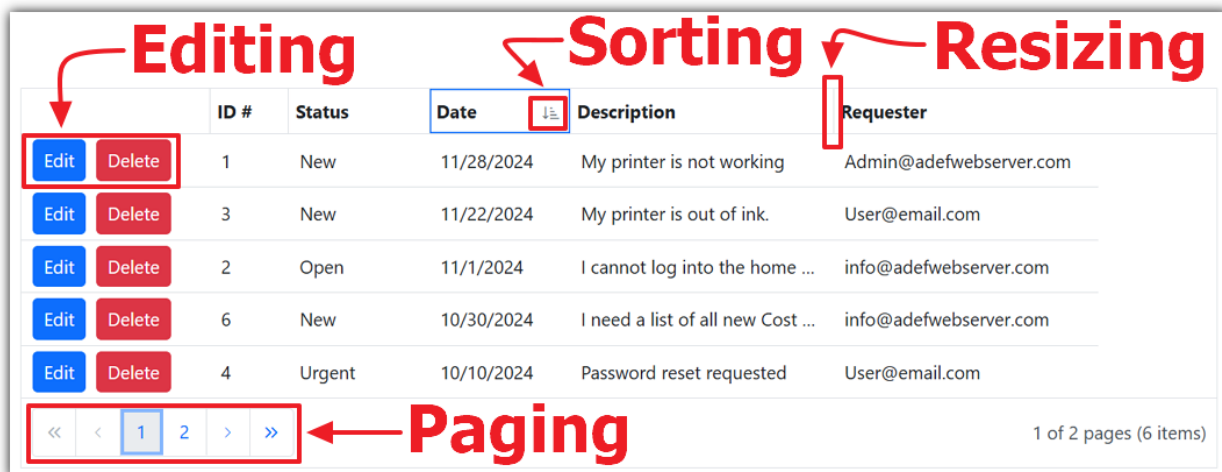


Figure 21: Syncfusion DataGrid

Clicking the Tickets link will display the Syncfusion DataGrid, which will allow the administrator to edit records and sort, page, and resize the DataGrid.

EDIT TICKET # 3

Ticket Status

New

Ticket Date

11/22/2024

Ticket Description

My printer is out of ink.

Requester Email

User@email.com

Date	Description
11/28/2024	What type of printer do you have?
11/28/2024	I have a HP101

New Help Desk Ticket Detail

Add Detail

Save

Figure 22: Syncfusion Dialog

Selecting the Edit button next to a record in the DataGrid will open the record in the Syncfusion Dialog control. This dialog will allow the administrator to edit all fields of the help desk ticket, as well as add ticket detail records at the bottom of the form. When the administrator saves the record, the user who created the help desk ticket receives an email with a link that navigates them to the ticket.

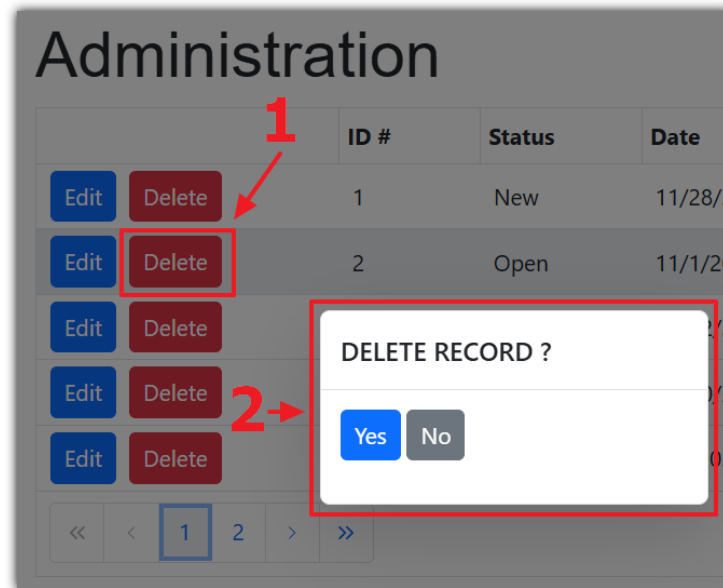


Figure 23: Delete confirmation

Clicking the Delete button next to a record in the DataGrid will open the delete confirmation pop-up window in the Syncfusion Dialog.

Clicking Yes will delete the record, and clicking No will cancel the action.

Chapter 3 Create the Help Desk Application

In this chapter, we will cover the steps to create the help desk application.

The screenshot displays a web application for managing help desk tickets. It features three main components:

- New Help Desk Ticket Form:** Located on the left, it includes a dropdown menu for ticket status (set to 'New'), a date field (11/11/2024), a text area for the ticket description ('My printer is not working'), and a text field for the requester email ('info@adefwebserver.com'). It has 'Save' and 'Smart Paste' buttons.
- Administration Table:** A central table listing tickets with columns for ID #, Status, Date, and Description. Each row has 'Edit' and 'Delete' buttons. The table shows 5 items, with the current page being 2 of 3 pages (12 items total).
- EDIT TICKET # 13 Modal:** A modal window on the right for editing ticket #13. It contains fields for Ticket Status (Open), Ticket Date (11/5/2024), Ticket Description (Allowed to use VPN), and Requester Email (admin@adefwebserver.com). It also has a 'Save' button.

ID #	Status	Date	Description
10	New	11/3/202...	This is ticket num
11	New	11/3/202...	This is ticket num
12	New	11/5/202...	Request to instal
13	New	11/5/202...	Allowed to use V
14	Open	11/5/202...	An urgent ticket

Figure 24: The help desk application in action

The source code for the completed application is available on [GitHub](#).

Install SQL Server

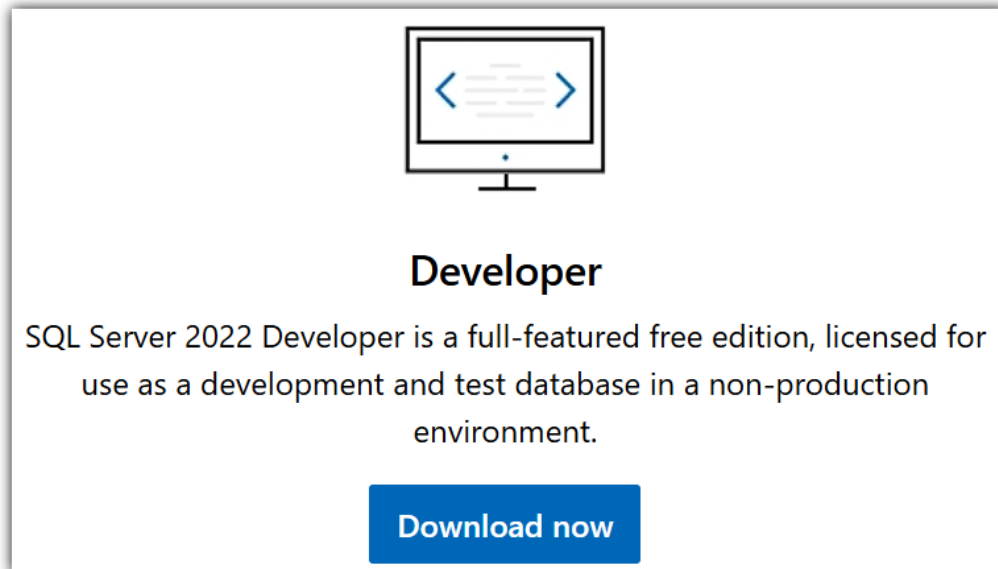


Figure 25: SQL Server

The application requires a database to store the data. You can download and install the free SQL Server 2022 Developer edition from the following link: <https://www.microsoft.com/en-us/sql-server/sql-server-downloads>. Of course, you can also use alternatives such as MySQL or SQLite.

Install .NET Core and Visual Studio

To create the application, these steps are required if you do not already have the following software installed:

1. Install the .NET 9.0 SDK (or later) from <https://dotnet.microsoft.com/download/dotnet-core>.
2. Install Visual Studio 2022 version 17.12.2 (or later), with the ASP.NET and web development workload from <https://visualstudio.microsoft.com/vs/>.



Note: The requirements for creating Blazor applications are constantly evolving. For the latest requirements, see the following documentation: <https://docs.microsoft.com/en-us/aspnet/core/blazor/get-started>.



Note: If you install Visual Studio 2022 and select the .NET Core workload during installation, the .NET SDK and runtime will be installed for you. See <https://docs.microsoft.com/en-us/dotnet/core/install/sdk?pivots=os-windows> for more information.

Create the project

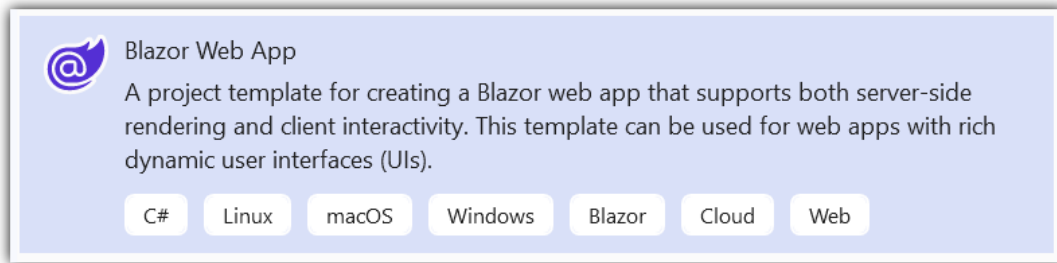


Figure 26: Blazor app

To create the project:

1. Open **Visual Studio** and select **Create a New Project**.
2. Select **Blazor Web App**, then select **Next**.

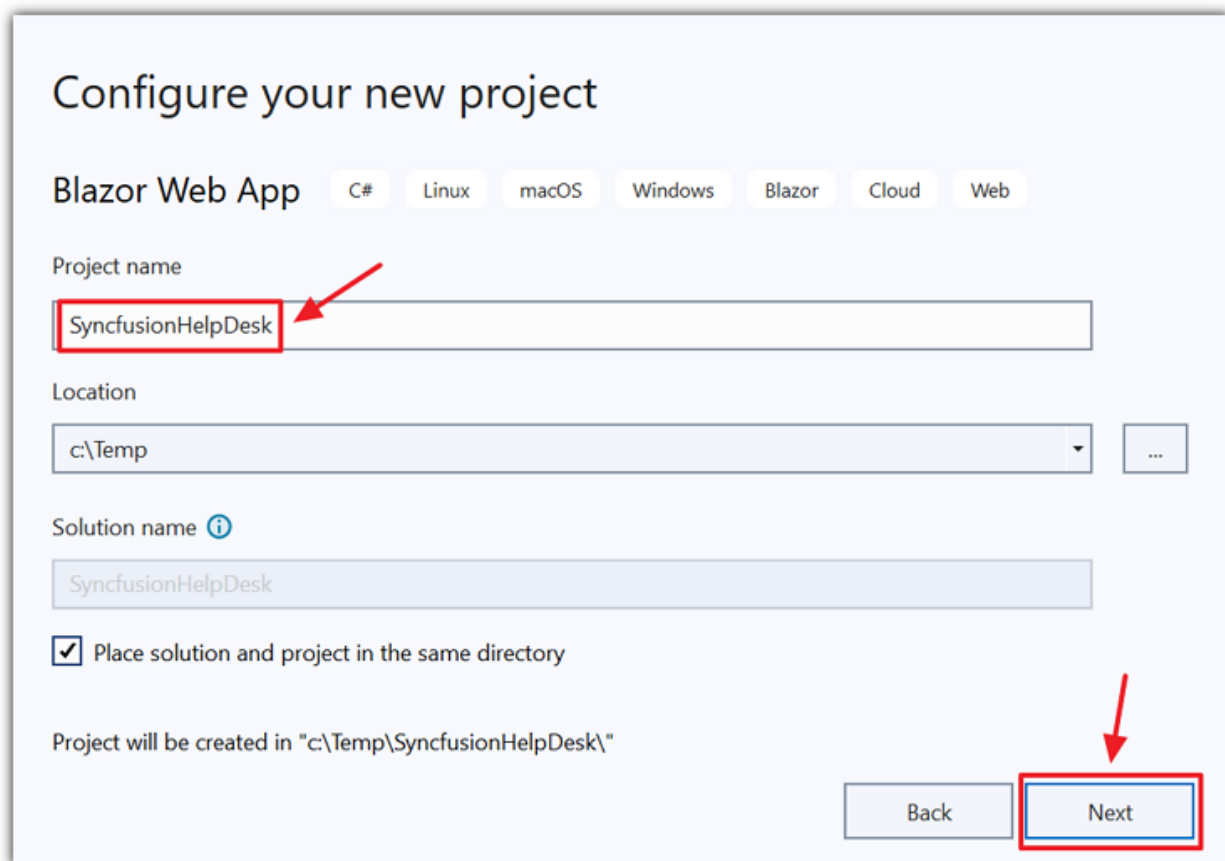


Figure 27: Set project name

3. Enter a **Project name** and select **Next**.
4. In the **Create a new Blazor app** dialog, select **Change** under **Authentication**.

Additional information

Blazor Web App C# Linux macOS Windows Blazor Cloud Web

Framework ⓘ
[.NET 9.0 (Standard Term Support)]

Authentication type ⓘ
[Individual Accounts]

☒ Configure for HTTPS ⓘ

Interactive render mode ⓘ
[Server]

Interactivity location ⓘ
[Global]

☒ Include sample pages ⓘ

☒ Do not use top-level statements ⓘ

☐ Enlist in .NET Aspire orchestration ⓘ

Back Create

Figure 28: Set Authentication

5. In the **Additional Information** dialog, select:
 - **.NET 9.0** for **Framework**.
 - **Individual User Accounts** for **Authentication type**.
 - **Server** for **Interactive render mode**.
 - **Global** for **Interactivity location**.
 - **Include sample pages** and **Do not use top-level statements**.
6. Select **Create**.

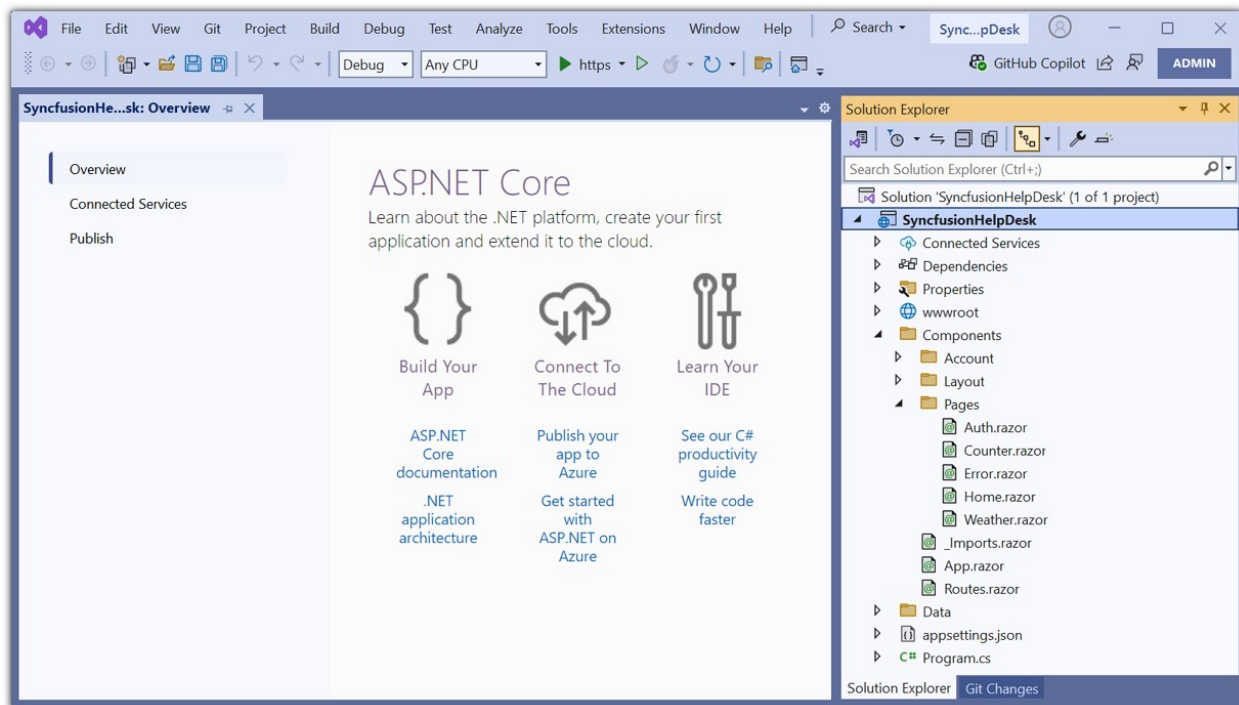


Figure 29: The new project in Visual Studio

The project will be created and opened in **Visual Studio**.

Create the database

1. From the toolbar, select **View > SQL Server Object Explorer**.

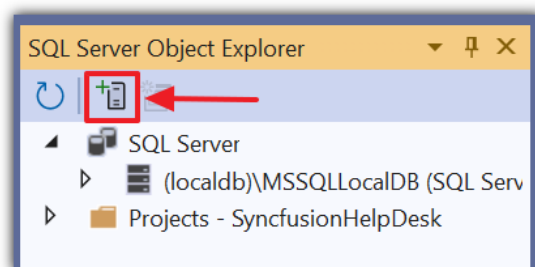


Figure 30: Add SQL Server

2. Select **Add SQL Server** to add a connection to your database server if it isn't already in the SQL Server list.



Note: For this example, we do not want to use the (localdb) connection for SQL Express, which you may see in the list.

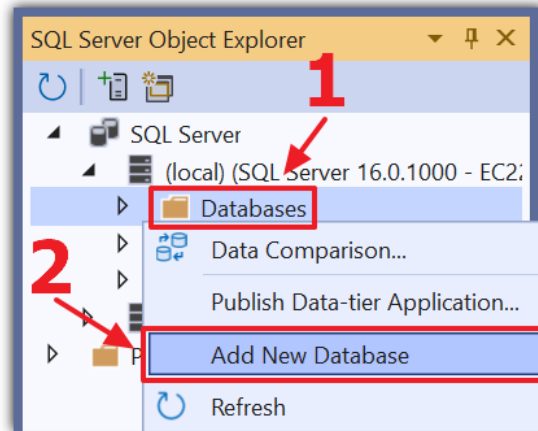


Figure 31: Add new database

- Expand the tree node for your SQL server, then right-click **Databases** and select **Add New Database**.
- Name the database **SyncfusionHelpDesk**.

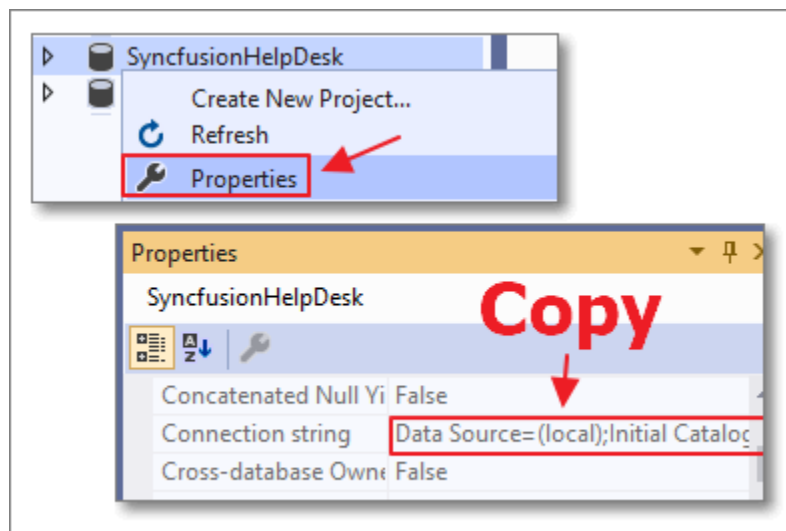


Figure 32: Copy connection string

- After the database has been created, right-click on it and select **Properties**. In the Properties window, copy the **Connection string**.

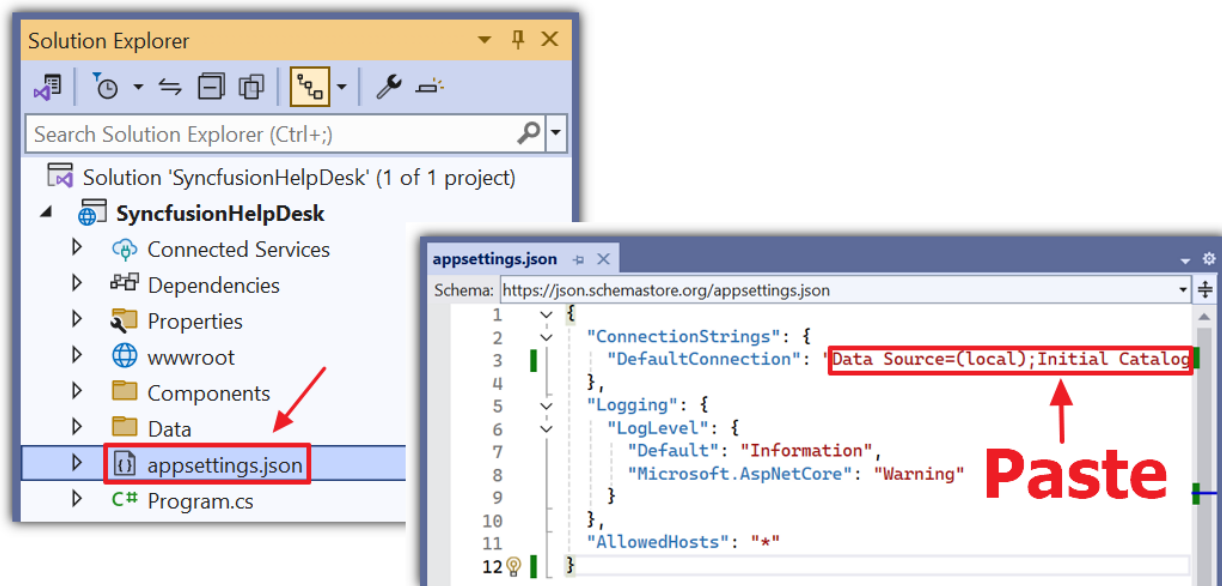


Figure 33: Paste connection string

6. Open the **appsettings.json** file and paste the connection string in the **DefaultConnection** property.
7. Save and close the file.

Enable role management

The default code the Visual Studio wizard creates will allow us to create new users. However, we want some users to be *administrators*. To do this, we must enable **role management** by following these steps:

1. Open the **Program.cs** file and replace the code from Code Listing 9 with the code in Code Listing 10 to enable role management.

Code Listing 9: Original identity code

```
builder.Services.AddAuthentication(options=>
{
options.DefaultScheme = IdentityConstants.ApplicationScheme;
options.DefaultSignInScheme = IdentityConstants.ExternalScheme;
}).AddIdentityCookies();
```

Code Listing 10: Updated identity code

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.SignIn.RequireConfirmedAccount = true;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddSignInManager()
.AddDefaultTokenProviders();
```

2. In **Visual Studio**, build the project and press **Ctrl+F5** to run the application in **Start without Debugging** mode. The application will open in your web browser.

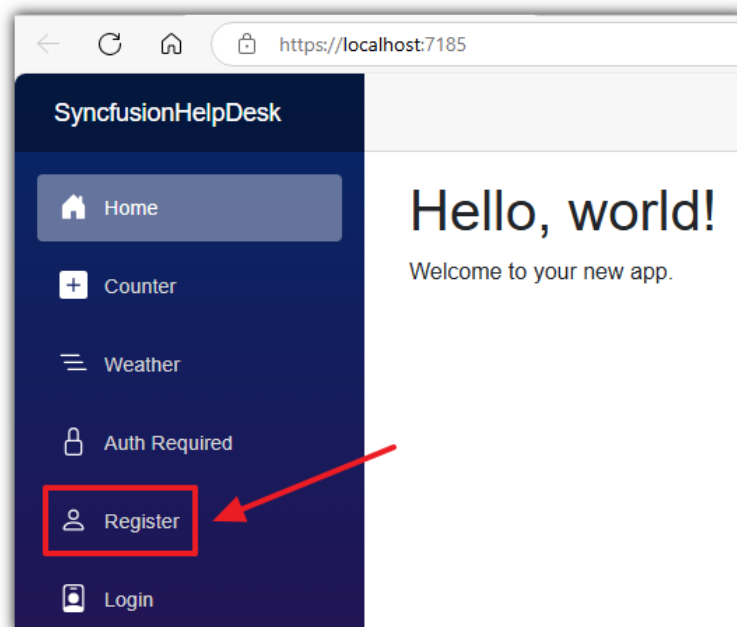


Figure 34: Register user

3. Go to **Register** and create a user. You will then see the following:

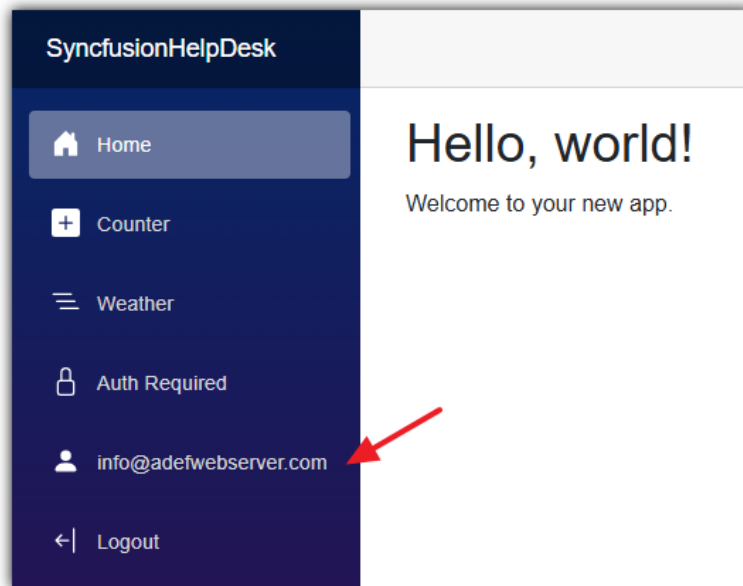


Figure 37: Logged in

The application will display the home page and indicate that you are logged in. From here, you can close the web browser to stop the application and return to **Visual Studio**.

Chapter 4 Explore the Project

In this chapter, we will explore the Blazor project we just created.

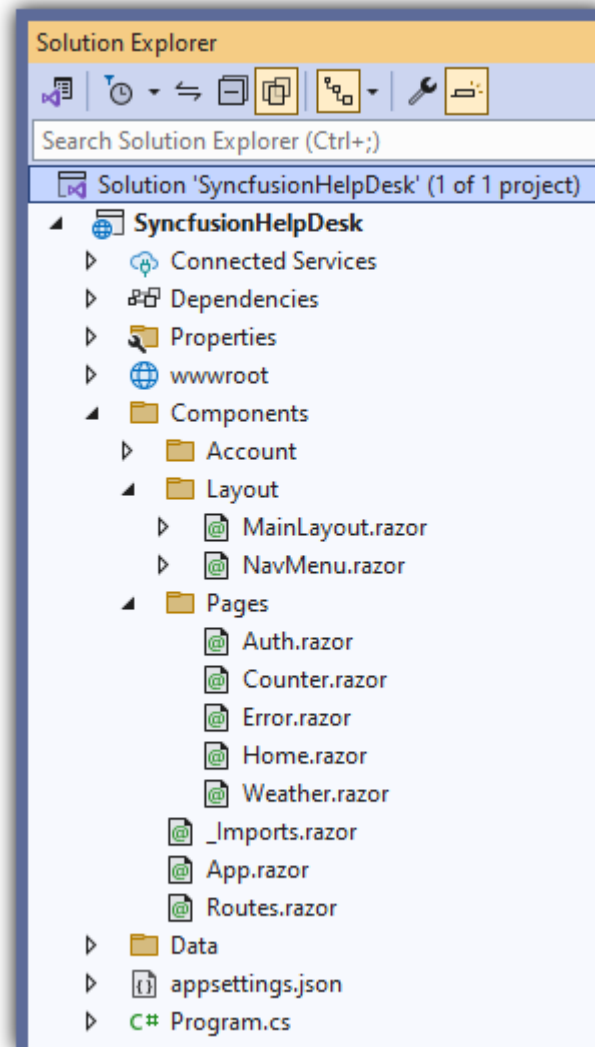


Figure 38: Explore the project

Startup

The Program.cs file is the entry point for the application.

Code Listing 11: Program.cs

```
app.MapRazorComponents<App>()  
    .AddInteractiveServerRenderMode();
```

This page specifies that the app component, contained in the App.razor file, is to be rendered as the root component of the application.

Code Listing 12: App.razor page

```
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
    <meta charset="utf-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0"  
/>  
    <base href="/" />  
    <link rel="stylesheet"  
href="@Assets["lib/bootstrap/dist/css/bootstrap.min.css"]" />  
    <link rel="stylesheet" href="@Assets["app.css"]" />  
    <link rel="stylesheet" href="@Assets["SyncfusionHelpDesk.styles.css"]"  
/>  
    <ImportMap />  
    <link rel="icon" type="image/png" href="favicon.png" />  
    <HeadOutlet @rendermode="PageRenderMode" />  
</head>  
  
<body>  
    <Routes @rendermode="PageRenderMode" />  
    <script src="_framework/blazor.web.js"></script>  
</body>  
  
</html>  
  
@code {  
    [CascadingParameter]  
    private HttpContext HttpContext { get; set; } = default!;  
  
    private IComponentRenderMode? PageRenderMode =>  
        HttpContext.AcceptsInteractiveRouting() ? InteractiveServer : null;  
}
```

This page also loads the blazor.web.js JavaScript file, which sets up the SignalR connection between the user's web browser and the server-side code.

Routing

Page request routing in the application is configured in the **Routes.razor** control.

Code Listing 13: Routes.razor

```
@using SyncfusionHelpDesk.Components.Account.Shared
<Router AppAssembly="typeof(Program).Assembly">
    <Found Context="routeData">
        <AuthorizeRouteView RouteData="routeData"
DefaultLayout="typeof(Layout.MainLayout)">
            <NotAuthorized>
                <RedirectToLogin />
            </NotAuthorized>
        </AuthorizeRouteView>
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
</Router>
```

This component intercepts web browser navigation and renders the layout and the target page after applying any configured authorization rules.

Layouts

The Routes.razor control specifies MainLayout (contained in the MainLayout.razor file) as the application's default layout component.

Code Listing 14: MainLayout.razor

```
@inherits LayoutComponentBase

<div class="page">
    <div class="sidebar">
        <NavMenu />
    </div>

    <main>
        <div class="top-row px-4">
            <a href="https://learn.microsoft.com/aspnet/core/"
target="_blank">About</a>
        </div>

        <article class="content px-4">
            @Body
        </article>
    </main>
</div>
```

```
<div id="blazor-error-ui" data-nosnippet>
  An unhandled error has occurred.
  <a href="." class="reload">Reload</a>
  <span class="dismiss">✕</span>
</div>
```

The MainLayout control inherits from LayoutComponentBase and will inject the content of the Razor page that the user is navigating to at the location of the **@Body** parameter in the template.

The remaining page markup, including the NavMenu control (located in the NavMenu.razor file), will be displayed around the content, creating a consistent page layout.

Chapter 5 Add Syncfusion

Syncfusion controls allow us to easily implement advanced functionality with a minimum amount of code. The Syncfusion controls are contained in NuGet packages. In this chapter, we will cover the steps to obtain the packages and configure them for our application.



Note: For the latest requirements to use Syncfusion, see the following link: <https://blazor.syncfusion.com/documentation/system-requirements/>.

Install NuGet packages

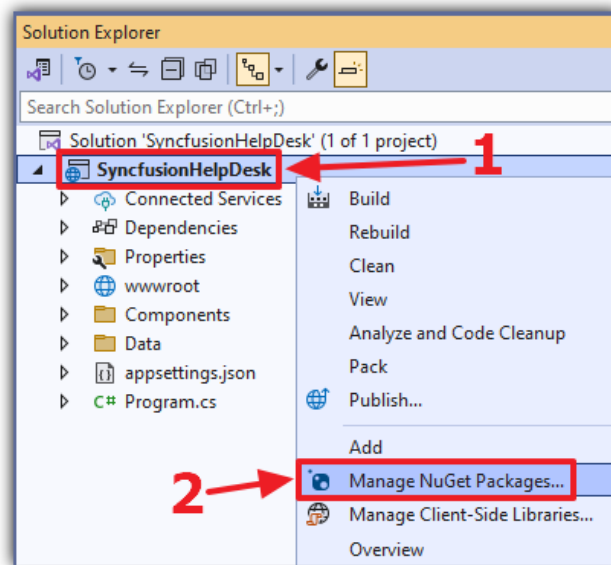


Figure 39: NuGet packages

1. Right-click on the project node and select **Manage NuGet Packages**.

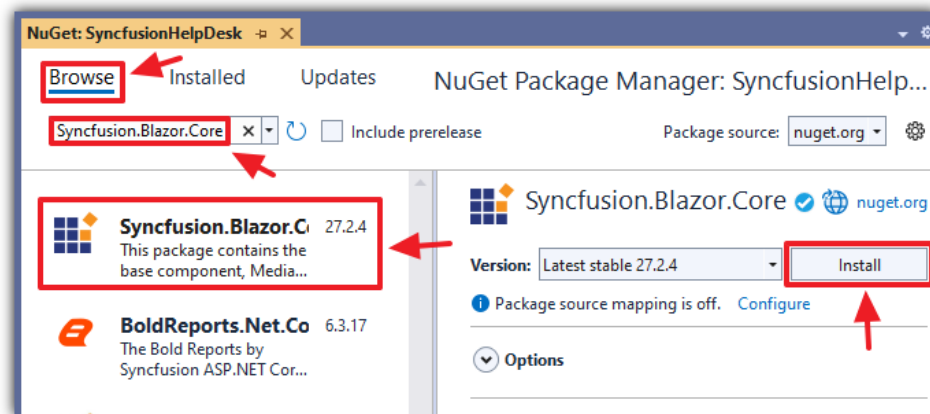


Figure 40: Install NuGet packages

2. Select the Browse tab and install the following NuGet packages:

- Syncfusion.Blazor.Core
- Syncfusion.Blazor.Calendars
- Syncfusion.Blazor.DataForm
- Syncfusion.Blazor.DropDowns
- Syncfusion.Blazor.Inputs
- Syncfusion.Blazor.Notifications
- Syncfusion.Blazor.SmartComponents
- Syncfusion.Blazor.Themes
- Syncfusion.Blazor.Grid
- System.Text.Json

Additional configuration

1. Open the **_Imports.razor** file and add the following code:

Code Listing 15: _Imports.razor

```
@using Syncfusion.Blazor
@using Syncfusion.Blazor.Calendars
@using Syncfusion.Blazor.DropDowns
@using Syncfusion.Blazor.Buttons
@using Syncfusion.Blazor.Inputs
@using Syncfusion.Blazor.Notifications
@using Syncfusion.Blazor.Popups
@using Syncfusion.Blazor.Grids
@using Syncfusion.Blazor.SmartComponents
```

2. Open the **Program.cs** file and add the following using statements:

Code Listing 15: Startup.cs using statement

```
using Syncfusion.Blazor;  
using Syncfusion.Blazor.SmartComponents;
```

3. Add the following to the **ConfigureServices** section, above the **var app = builder.Build();** line:

Code Listing 16: Startup.cs AddSyncfusionBlazor

```
builder.Services.AddSyncfusionBlazor();
```

4. Add the following to the **<head>** element of the **App.razor** page:

Code Listing 17: Syncfusion style link

```
<link href="_content/Syncfusion.Blazor.Themes/bootstrap5.css"  
rel="stylesheet" />
```

5. Finally, add the following to the **<body>** element of the **App.razor** page:

Code Listing 18: Syncfusion JavaScript link

```
<script src="_content/Syncfusion.Blazor.Core/scripts/syncfusion-  
blazor.min.js" type="text/javascript"></script>
```



Note: When you run the application, you will see a message:

This application was built using a trial version of Syncfusion Essential Studio. Please include a valid license to permanently remove this license validation message. You can also obtain a free 30-day evaluation license to temporarily remove this message during the evaluation period. Please refer to this [help topic](#) for more information.

Click the link in the message for instructions on obtaining a key to make the message go away.

Chapter 6 Creating a Data Layer

We will now add additional tables to the database to support the custom code we plan to write. To allow our code to communicate with these tables, we require a data layer.

This data layer will also allow us to organize and reuse code efficiently.

Create the database tables

The first step is to add the database tables we will need.

1. In the **SQL Server Object Explorer**, right-click on the database and select **New Query**.

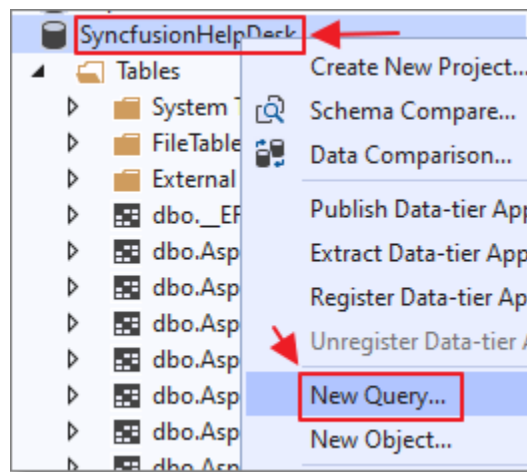


Figure 41: New Query

2. Enter the following script:

Code Listing 19: SQL script

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[HelpDeskTicketDetails](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [HelpDeskTicketId] [int] NOT NULL,
    [TicketDetailDate] [datetime] NOT NULL,
    [TicketDescription] [nvarchar](max) NOT NULL,
    CONSTRAINT [PK_HelpDeskTicketDetails] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)
```

```

)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[HelpDeskTickets](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [TicketStatus] [nvarchar](50) NOT NULL,
    [TicketDate] [datetime] NOT NULL,
    [TicketDescription] [nvarchar](max) NOT NULL,
    [TicketRequesterEmail] [nvarchar](500) NOT NULL,
    [TicketGUID] [nvarchar](500) NOT NULL,
    CONSTRAINT [PK_HelpDeskTickets] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO
ALTER TABLE [dbo].[HelpDeskTicketDetails]
WITH CHECK
ADD CONSTRAINT [FK_HelpDeskTicketDetails_HelpDeskTickets]
FOREIGN KEY([HelpDeskTicketId])
REFERENCES [dbo].[HelpDeskTickets] ([Id])
ON DELETE CASCADE
GO
ALTER TABLE [dbo].[HelpDeskTicketDetails]
CHECK CONSTRAINT [FK_HelpDeskTicketDetails_HelpDeskTickets]
GO

```

3. Select the **Execute** icon.

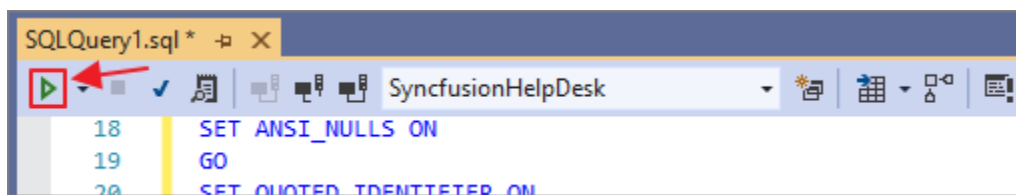


Figure 42: Click execute

4. Close the window and refresh the view of the tables in the database.

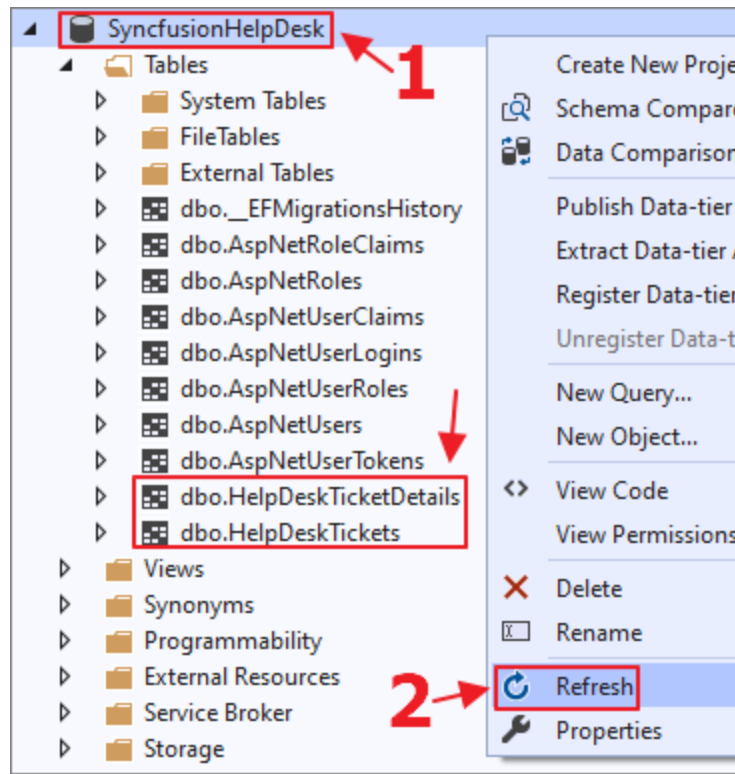


Figure 43: Refresh database

You will see that the new tables have been added.

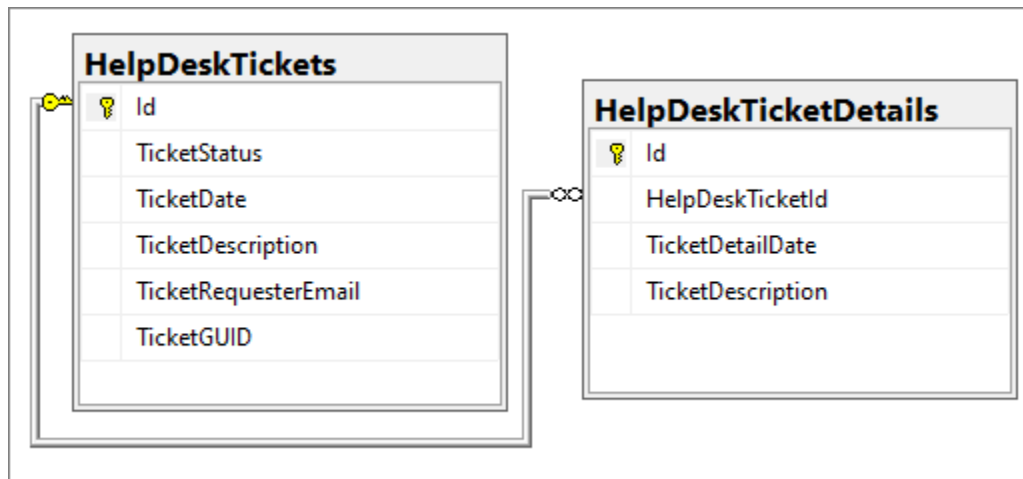


Figure 44: Database diagram

The diagram in Figure 44 shows the one-to-many relationship between the newly added tables. A HelpDeskTickets record is created first. As the issue is processed, multiple associated HelpDeskTicketDetails records are added.

Create the DataContext by using EF Core tools

We will now create the DataContext code that will allow the SyncfusionHelpDeskService, which will be created in later steps, to communicate with the database tables we just added.

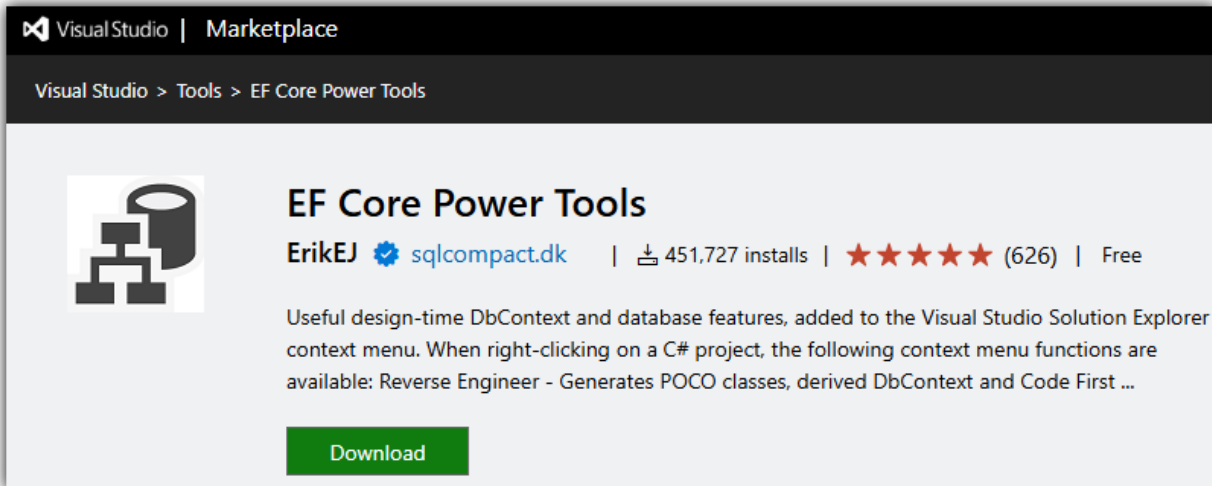


Figure 45: EF Core Power Tools

To create the DataContext code:

1. Install EF Core Power Tools from the following link:
<https://marketplace.visualstudio.com/items?itemName=ErikEJ.EFCorePowerTools>.

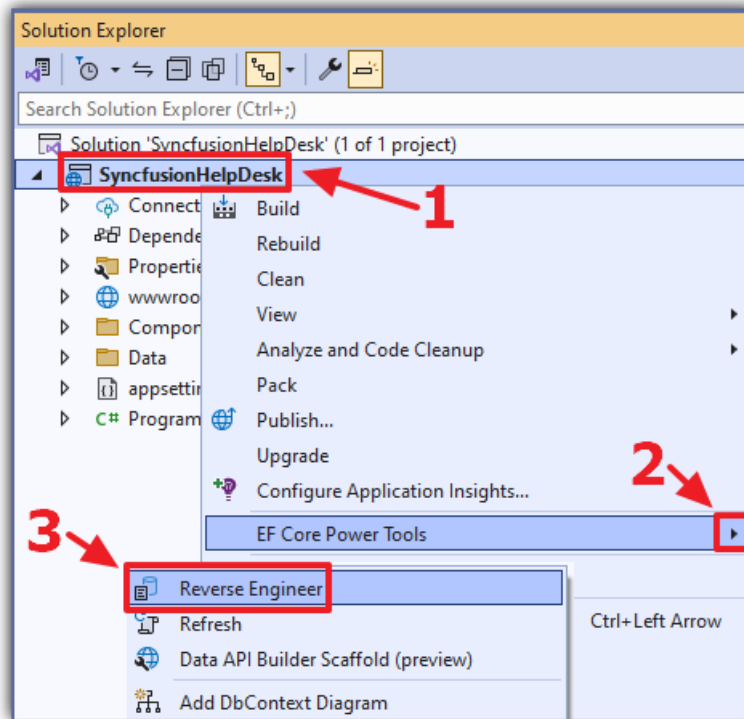


Figure 46: EF Core Power Tools

2. Right-click on the project node in the **Solution Explorer** and select **EF Core Power Tools > Reverse Engineer**.

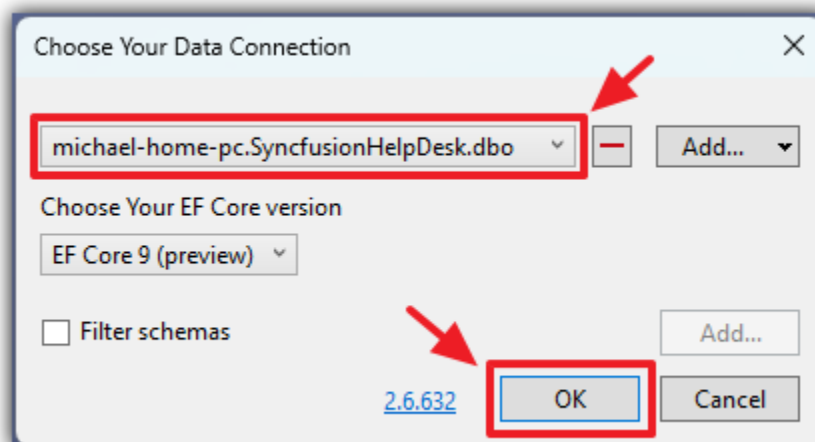


Figure 47: Create a connection

3. Select **Add** to create a connection to the database if one does not already exist in the dropdown.
4. Select the database connection in the dropdown and click **OK**.

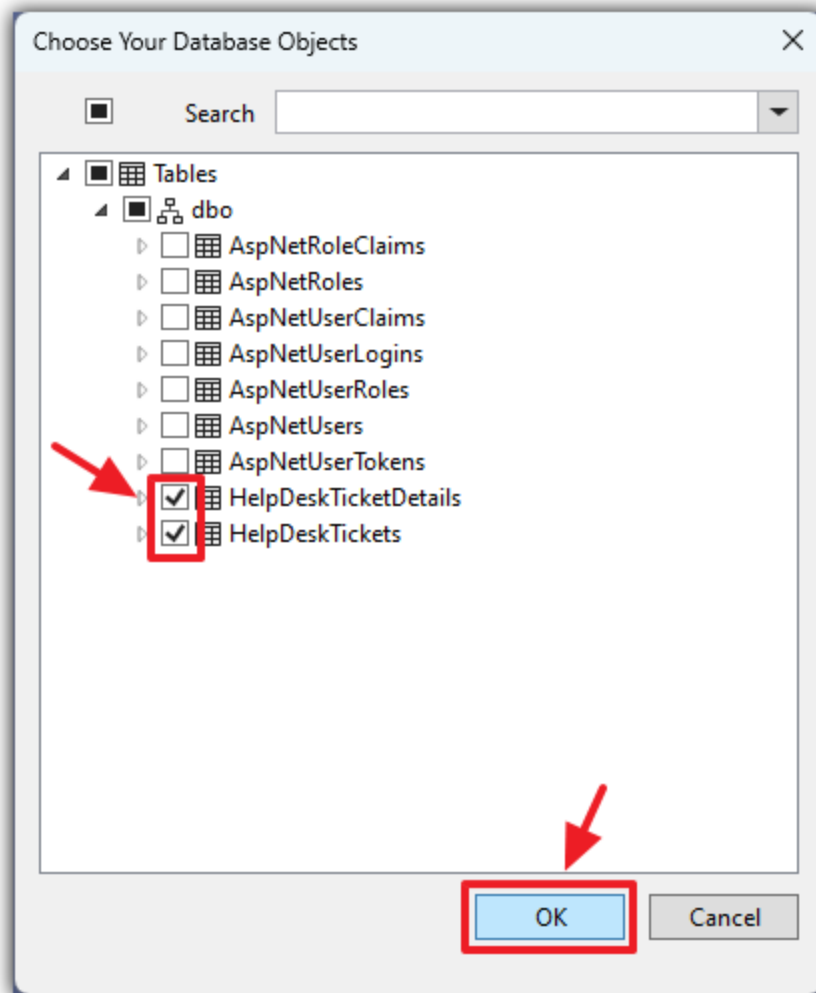


Figure 48: Select tables

5. Select the **HelpDeskTicketDetails** table and the **HelpDeskTickets** table. Then, select **OK**.

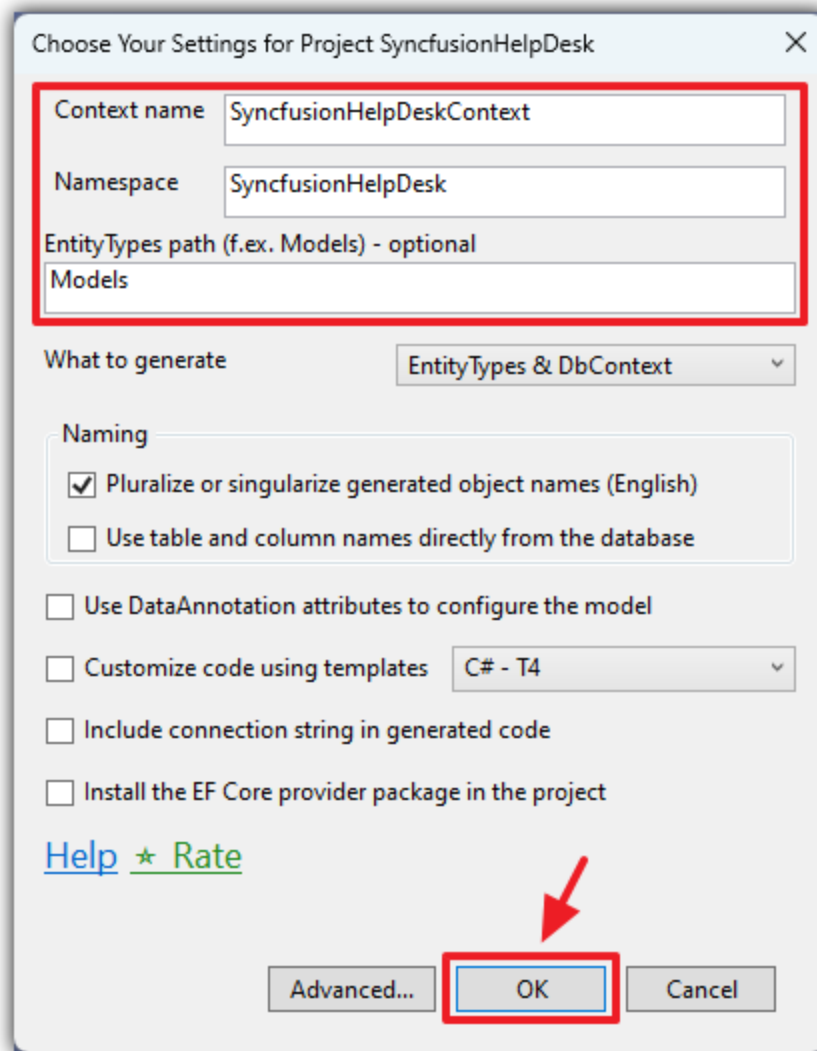


Figure 49: Configure DataContext

6. Enter the following values:
 - **Context name:** SyncfusionHelpDeskContext
 - **Namespace:** SyncfusionHelpDesk
 - **EntityTypes path:** Models
7. Select **OK**.

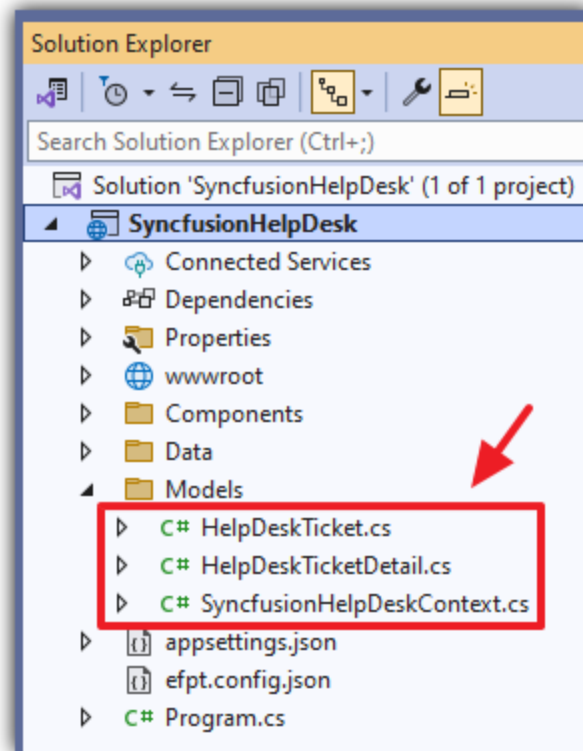


Figure 50: DataContext created

In the Solution Explorer, you will see that the DataContext has been created.

Set the database connection

The DataContext code needs the connection to the database to be set.

1. Open the **Programs.cs** file and add the following code to the **ConfigureServices** section before the **var app = builder.Build();** line:

Code Listing 20: Database connection

```
// To access HelpDesk tables

builder.Services.AddDbContextFactory<SyncfusionHelpDeskContext>(options =>
    options.UseSqlServer(connectionString));
```

2. Save the file.
3. Select **Build > Rebuild Solution**. The application should build without any errors.

Create the SyncfusionHelpDeskService

We will now create the service that will provide all the remaining data access methods we will need for the application.

1. Create a new folder called **Services**.
2. In the **Services** folder, add a new file called **SyncfusionHelpDeskService.cs** with the following code:

Code Listing 21: SyncfusionHelpDeskService

```
#nullable disable
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using SyncfusionHelpDesk.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace SyncfusionHelpDesk.Data
{
    public class SyncfusionHelpDeskService : IDisposable
    {
        SyncfusionHelpDeskContext syncfusionHelpDeskContext;
        public SyncfusionHelpDeskService() { }

        public IQueryable<HelpDeskTicket>
            GetHelpDeskTickets(
                IDbContextFactory<SyncfusionHelpDeskContext> dbContextFactory,
                bool IsAdmin,
                string paramEmail)
        {
            // Return all HelpDesk Tickets as IQueryable
            // SfGrid will use this to only pull records
            // for the page that it is currently displaying
            // Note: AsNoTracking() is used because it is
            // quicker to execute and we do not need
            // Entity Framework change tracking at this point

            syncfusionHelpDeskContext = dbContextFactory.CreateDbContext();

            if (IsAdmin)
            {
                // Admin User
                return
                syncfusionHelpDeskContext.HelpDeskTickets.AsNoTracking();
            }
            else
            {
            }
        }
    }
}
```

```

        {
            // Regular User
            return syncfusionHelpDeskContext.HelpDeskTickets
                .Where(x => x.TicketRequesterEmail == paramEmail)
                .AsNoTracking();
        }
    }

    public async Task<HelpDeskTicket>
        GetHelpDeskTicketAsync(
            IDbContextFactory<SyncfusionHelpDeskContext> dbContextFactory,
            string HelpDeskTicketGuid)
    {
        // Get the existing record
        syncfusionHelpDeskContext = dbContextFactory.CreateDbContext();

        var ExistingTicket = await
syncfusionHelpDeskContext.HelpDeskTickets
            .Include(x => x.HelpDeskTicketDetails)
            .Where(x => x.TicketGuid == HelpDeskTicketGuid)
            .AsNoTracking()
            .FirstOrDefaultAsync();

        return ExistingTicket;
    }

    public Task<HelpDeskTicket>
        CreateTicketAsync(
            IDbContextFactory<SyncfusionHelpDeskContext> dbContextFactory,
            HelpDeskTicket newHelpDeskTickets)
    {
        // Add a new Help Desk Ticket
        syncfusionHelpDeskContext = dbContextFactory.CreateDbContext();

syncfusionHelpDeskContext.HelpDeskTickets.Add(newHelpDeskTickets);
        syncfusionHelpDeskContext.SaveChanges();

        return Task.FromResult(newHelpDeskTickets);
    }

    public Task<bool>
        UpdateTicketAsync(
            IDbContextFactory<SyncfusionHelpDeskContext> dbContextFactory,
            HelpDeskTicket UpdatedHelpDeskTickets)
    {
        // Get the existing record
        syncfusionHelpDeskContext = dbContextFactory.CreateDbContext();

```

```

var ExistingTicket =
    syncfusionHelpDeskContext.HelpDeskTickets
        .Where(x => x.Id == UpdatedHelpDeskTickets.Id)
        .FirstOrDefault();

if (ExistingTicket != null)
{
    ExistingTicket.TicketDate =
        UpdatedHelpDeskTickets.TicketDate;

    ExistingTicket.TicketDescription =
        UpdatedHelpDeskTickets.TicketDescription;

    ExistingTicket.TicketGuid =
        UpdatedHelpDeskTickets.TicketGuid;

    ExistingTicket.TicketRequesterEmail =
        UpdatedHelpDeskTickets.TicketRequesterEmail;

    ExistingTicket.TicketStatus =
        UpdatedHelpDeskTickets.TicketStatus;

    // Insert any new TicketDetails
    if (UpdatedHelpDeskTickets.HelpDeskTicketDetails != null)
    {
        foreach (var item in
            UpdatedHelpDeskTickets.HelpDeskTicketDetails)
        {
            if (item.Id == 0)
            {
                // Create New HelpDeskTicketDetails record
                HelpDeskTicketDetail newHelpDeskTicketDetails =
                    new HelpDeskTicketDetail();
                newHelpDeskTicketDetails.HelpDeskTicketId =
                    UpdatedHelpDeskTickets.Id;
                newHelpDeskTicketDetails.TicketDetailDate =
                    DateTime.Now;
                newHelpDeskTicketDetails.TicketDescription =
                    item.TicketDescription;

                syncfusionHelpDeskContext.HelpDeskTicketDetails
                    .Add(newHelpDeskTicketDetails);
            }
        }
    }

    syncfusionHelpDeskContext.SaveChanges();
}
else

```



```

        {
            return Task.FromResult(false);
        }

        return Task.FromResult(true);
    }

    public Task<bool>
        DeleteHelpDeskTicketsAsync(
            IDbContextFactory<SyncfusionHelpDeskContext> dbContextFactory,
            HelpDeskTicket DeleteHelpDeskTickets)
    {
        // Get the existing record
        syncfusionHelpDeskContext = dbContextFactory.CreateDbContext();

        var ExistingTicket =
            syncfusionHelpDeskContext.HelpDeskTickets
                .Include(x => x.HelpDeskTicketDetails)
                .Where(x => x.Id == DeleteHelpDeskTickets.Id)
                .FirstOrDefault();

        if (ExistingTicket != null)
        {
            // Delete the Help Desk Ticket

            syncfusionHelpDeskContext.HelpDeskTickets.Remove(ExistingTicket);
            syncfusionHelpDeskContext.SaveChanges();
        }
        else
        {
            return Task.FromResult(false);
        }

        return Task.FromResult(true);
    }

    // Ensures the context is disposed when the component is disposed
    public void Dispose() => syncfusionHelpDeskContext?.Dispose();
}

```

Register the SyncfusionHelpDeskService

Finally, we need to register this service so that we can make it available to our code pages.

Open the **Program.cs** file and add the following code to the **ConfigureServices** section (before the **var app = builder.Build();** line:

Code Listing 22: SyncfusionHelpDeskService

```
builder.Services.AddScoped<SyncfusionHelpDeskService>();
```

We will later inject this service into our .razor code.

Create the administrator

We will now write code to programmatically create an administrator role and a button to add the current user to it.

1. Add the following method to the **Program** class in the **Program.cs** file:

Code Listing 23: Role creation logic

```
// Role creation logic
private static async Task CreateRoles(
    IServiceProvider serviceProvider)
{
    using var scope =
        serviceProvider.CreateScope();
    var roleManager =
        scope.ServiceProvider
            .GetRequiredService<RoleManager<IdentityRole>>();

    // Check if the Administrator role exists, if not, create it
    if (!await roleManager.RoleExistsAsync("Administrators"))
    {
        var adminRole = new IdentityRole("Administrators");
        await roleManager.CreateAsync(adminRole);
    }
}
```

This will create the administrator role if it does not already exist.

2. To call this method in **Program.cs**, change the code in Code Listing 25 to the code in Code Listing 26.

Code Listing 24: Original Main method

```
public static void Main(string[] args)
```

Code Listing 25: Updated Main method

```
public static async Task Main(string[] args)
```

3. Change the existing **app.Run()**; line in the **Program.cs** file to the following to call the new **CreateRoles** method and to run App.razor as async:

Code Listing 26: Call CreateRoles

```
// Ensure the Administrator role is created at startup
await CreateRoles(app.Services);

await app.RunAsync();
```

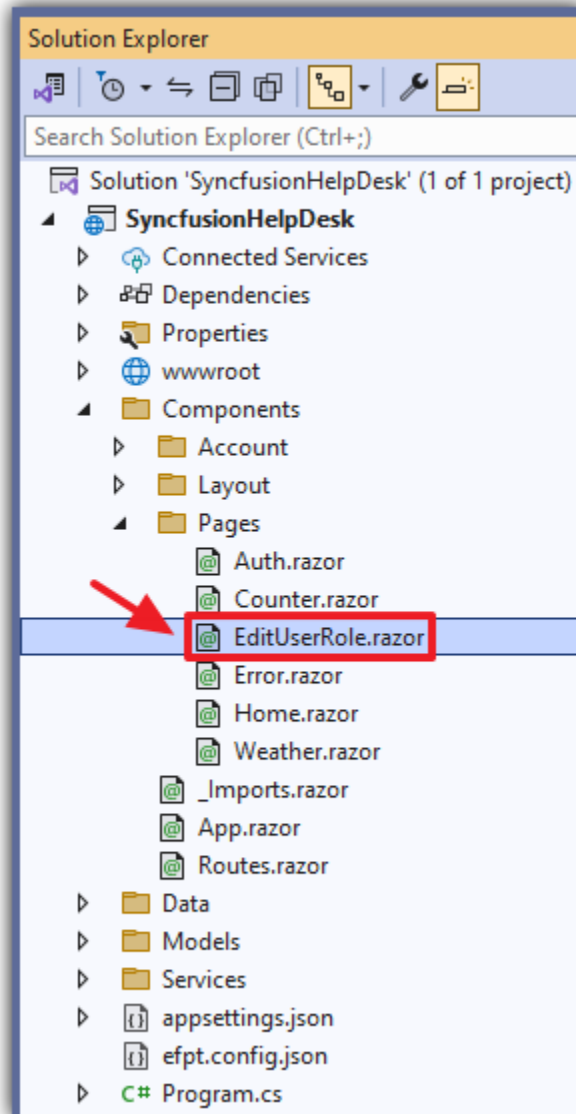


Figure 51: Create EditUserRole.razor page

We will now create a button that will allow the logged-in user to switch between being an administrator and a normal user.

1. Create a new Razor control called **EditUserRole.razor** with the following code:

Code Listing 27: EditUserRole code

```
@using Microsoft.AspNetCore.Identity
@using SyncfusionHelpDesk.Data
@inject RoleManager<IdentityRole> RoleManager
@inject UserManager<ApplicationUser> UserManager
@inject AuthenticationStateProvider AuthenticationStateProvider
@inject NavigationManager NavigationManager

@if (isLoggedIn)
{
    <div class="button-container">
        <SfButton OnClick="@ToggleRole"
                CssClass="e-secondary" Content="@buttonText">
        </SfButton>&nbsp;
    </div>
}
<br />
@code {
#nullable disable
    [Parameter] // Expose event to parent
    public EventCallback<bool> OnRoleChanged { get; set; }

    private string ADMINISTRATION_ROLE = "Administrators";
    private string buttonText = "Loading...";
    private ApplicationUser currentUser;
    private bool isAdmin;
    private bool isLoggedIn;

    protected override async Task OnInitializedAsync()
    {
        await LoadUserAndRole();
    }

    private async Task LoadUserAndRole()
    {
        var authState =
            await AuthenticationStateProvider
                .GetAuthenticationStateAsync();

        var user = authState.User;

        isLoggedIn = user.Identity?.IsAuthenticated == true;

        if (isLoggedIn)
        {
            currentUser =
                await UserManager.FindByNameAsync(user.Identity.Name);
        }
    }
}
```

```

        isAdmin =
        await UserManager
        .IsInRoleAsync(currentUser, ADMINISTRATION_ROLE);

        UpdateButtonText();
    }
}

private void UpdateButtonText()
{
    buttonText =
    (isAdmin == true)
    ? "Make me a normal user"
    : "Make me an administrator";
}

private async Task ToggleRole()
{
    if (currentUser != null)
    {
        if (isAdmin)
        {
            await UserManager.RemoveFromRoleAsync(
                currentUser, ADMINISTRATION_ROLE);

            isAdmin = false;
        }
        else
        {
            await UserManager.AddToRoleAsync(
                currentUser, ADMINISTRATION_ROLE);

            isAdmin = true;
        }

        UpdateButtonText();

        // Raise event to notify parent
        await OnRoleChanged.InvokeAsync();
    }
}
}

```

2. Open the **Home.razor** page, which is the home page of the application, and replace all the code with the following code:

Code Listing 28: Add control to Home.razor

```
@page "/"
```

```

@using Microsoft.EntityFrameworkCore
@using SyncfusionHelpDesk.Data;
@using SyncfusionHelpDesk.Models
@inject IDbContextFactory<SyncfusionHelpDeskContext> DbFactory
@inject SyncfusionHelpDeskService SyncfusionHelpDeskService
@inject NavigationManager NavigationManager

<EditUserRole OnRoleChanged="HandleRoleChange" />

@code {
#nullable disable

    private void HandleRoleChange()
    {
        // Reload the entire page
        NavigationManager.NavigateTo("/", true);
    }
}

```

3. Save the page and run the application. The home page of the application displays an empty page. However, we can click the login link to log in to the application.

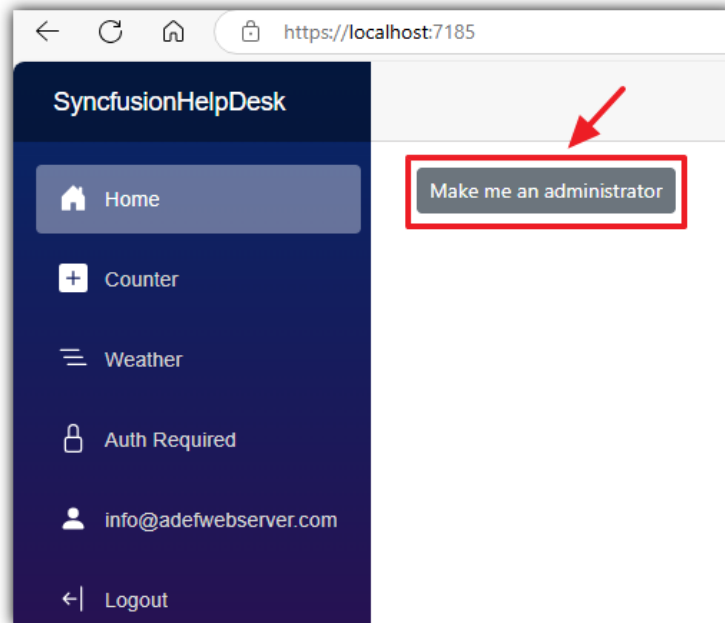


Figure 52: Button to allow the user to be an administrator

When we log in, the code on the Home.razor page runs the code that's contained in EditUserRole.razor to display a new button, which will make the currently logged-in user an administrator.

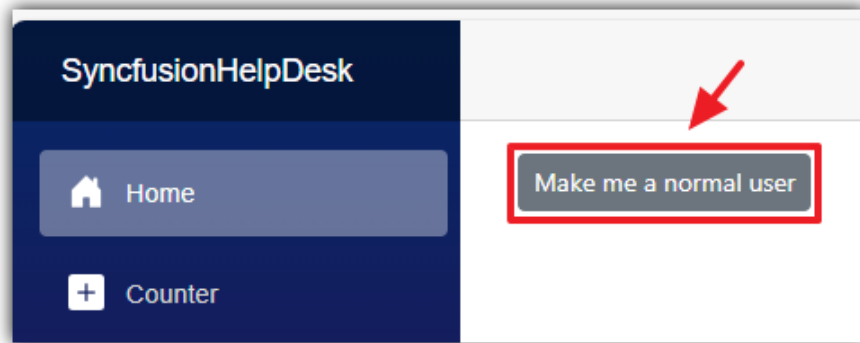


Figure 53: Administrator created

After the user has been made an administrator, the code detects this and displays a button to allow the user to change back to a normal user.

Chapter 7 Creating New Tickets

In this chapter, we will create a page containing a form to create help desk tickets.

Blazor Toast component

We will use the Syncfusion Blazor Toast component to display a brief pop-up message when a help desk ticket is submitted. To enable this, add the following code markup:

Code Listing 29: Toast control

```
<SfToast ID="toast_default"
  @ref="ToastObj"
  Title="Help Desk Ticket"
  Content="@ToastContent" Timeout="5000">
  <ToastPosition X="Right"></ToastPosition>
</SfToast>
```

Next, add the following to the **@code** section:

Code Listing 30: Toast properties

```
SfToast ToastObj;
private string ToastContent { get; set; } = "";
```

The **ToastObj** will provide programmatic access to the control, to open it and close it for example, and the **ToastContent** property will allow us to set the text that is displayed in the control.

Forms and validation

Blazor provides a method for you to create forms with validation to collect data.

Forms

Blazor provides an **EditForm** control that allows us to validate a form using data annotations. These data annotations are defined in a class that is specified in the **Model** property of the **EditForm** control.

Validation

The EditForm control defines a method to handle **OnValidSubmit**. This method is triggered only when the data in the form satisfies all the validation rules defined by the data annotations. Any validation errors are displayed using the **DataAnnotationsValidator**, the **ValidationSummary** control, or both.

HelpDeskTicket class

To support the forms and validation that we will implement:

1. Create a new folder named **Models**, add the following class that will be bound to the form, and provide the validation attributes:

Code Listing 31: HelpDeskTicketDTO.cs

```
#nullable disable
using System.ComponentModel.DataAnnotations;

namespace SyncfusionHelpDesk.Models
{
    public class HelpDeskTicketDTO
    {
        public int Id { get; set; }

        [Required]
        public string TicketStatus { get; set; }

        [Required]
        public DateTime TicketDate { get; set; }

        [Required(ErrorMessage =
            "The Ticket Description field is required.")]
        [StringLength(50, MinimumLength = 2,
            ErrorMessage =
                "Description must be a minimum of 2 " +
                "and maximum of 50 characters.")]
        public string TicketDescription { get; set; }

        [Required(ErrorMessage =
            "The Ticket Requester Email field " +
            "is required.")]
        [EmailAddress(ErrorMessage =
            "Must use a valid email address.")]
        public string TicketRequesterEmail { get; set; }

        public string TicketGuid { get; set; }
    }
}
```

Syncfusion Blazor controls

We will also employ the following Syncfusion controls in our form:

- **TextBox:** Allows us to gather data from the user and later display existing data.
- **DatePicker:** Allows the user to enter a date value and display an existing date value later.
- **Dropdown List:** Presents a list of predefined values that allows the user to choose a single option. In our example, we'll use a list of ticket status values. Later, we will use this control to also display an existing selected value.

The Dropdown List control requires a data collection for the display options. To enable this, go to the **Models** folder and add the following class:

Code Listing 32: HelpDeskStatus.cs

```
#nullable disable
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace SyncfusionHelpDesk.Data
{
    public class HelpDeskStatus
    {
        public string ID { get; set; }
        public string Text { get; set; }
    }

    public static class HelpDeskStatusData
    {
        public static List<HelpDeskStatus>
            Statuses { get; } = new()
        {
            new HelpDeskStatus {
                ID = "New", Text = "New" },
            new HelpDeskStatus {
                ID = "Open", Text = "Open" },
            new HelpDeskStatus {
                ID = "Urgent", Text = "Urgent" },
            new HelpDeskStatus {
                ID = "Closed", Text = "Closed" }
        };
    }
}
```



Note: You can get more information on the Syncfusion Blazor controls at <https://blazor.syncfusion.com/>.

New ticket form

We will now add the code to display the form.

1. Add the following markup to **Home.razor**:

Code Listing 33: Help desk form

```
<h3>New Help Desk Ticket</h3>
<br />
<EditForm ID="new-ticket-form" Model="@objHelpDeskTicket"
    OnValidSubmit="@HandleValidSubmit">
    <DataAnnotationsValidator></DataAnnotationsValidator>
    <div>
        <SfDropDownList TItem="HelpDeskStatus" TValue="string"
            PopupHeight="230px" Placeholder="Ticket Status"
            @bind-Value="@objHelpDeskTicket.TicketStatus"
            DataSource="@HelpDeskStatusData.Statuses">
            <DropDownListEvents TItem="HelpDeskStatus" TValue="string" />
            <DropDownListFieldSettings Text="Text" Value="ID" />
        </SfDropDownList>
    </div>
    <div>
        <SfDatePicker ID="TicketDate" Placeholder="Ticket Date"
            FloatLabelType="@FloatLabelType.Always"
            @bind-Value="@objHelpDeskTicket.TicketDate"
            Max="DateTime.Now"
            ShowClearButton="false"></SfDatePicker>
        <ValidationMessage For="@(() => objHelpDeskTicket.TicketDate)" />
    </div>
    <div>
        <SfTextBox Placeholder="Ticket Description"
            FloatLabelType="@FloatLabelType.Always"
            @bind-Value="@objHelpDeskTicket.TicketDescription">
        </SfTextBox>
        <ValidationMessage For="@(() =>
objHelpDeskTicket.TicketDescription)" />
    </div>
    <div>
        <SfTextBox Placeholder="Requester Email"
            FloatLabelType="@FloatLabelType.Always"
            @bind-Value="@objHelpDeskTicket.TicketRequesterEmail">
        </SfTextBox>
    </div>
```

```

        <ValidationMessage For="@(() =>
objHelpDeskTicket.TicketRequesterEmail)" />
    </div>
    <br />
    <div class="e-footer-content">
        <div class="button-container">
            <SfButton CssClass="e-normal e-primary" IsPrimary="true"
                Type="Submit">Save</SfButton>
        </div>
    </div>
</EditForm>

```

2. Add the following to the **@code** section:

Code Listing 34: HelpDeskTicket property

```

// Global property for the Help Desk Ticket
HelpDeskTicketDTO objHelpDeskTicket =
    new HelpDeskTicketDTO()
    {
        TicketDate = DateTime.Now,
        TicketStatus = "New"
    };

```

3. Finally, add the following code to insert the data into the database by calling the **CreateTicketAsync** method of the **SyncfusionHelpDeskService** when the form has successfully passed validation:

Code Listing 35: HandleValidSubmit

```

public async Task HandleValidSubmit(EditContext context)
{
    try
    {
        // Create a HelpDeskTickets
        HelpDeskTicket NewHelpDeskTickets =
            new HelpDeskTicket();

        // Set the values to the values entered
        // in the form
        NewHelpDeskTickets.TicketDate =
            objHelpDeskTicket.TicketDate;
        NewHelpDeskTickets.TicketDescription =
            objHelpDeskTicket.TicketDescription;
        NewHelpDeskTickets.TicketRequesterEmail =
            objHelpDeskTicket.TicketRequesterEmail;
        NewHelpDeskTickets.TicketStatus =
            objHelpDeskTicket.TicketStatus;

        NewHelpDeskTickets.HelpDeskTicketDetails =

```

```

        new List<HelpDeskTicketDetail>();

        // Create a new GUID for this Help Desk Ticket
        NewHelpDeskTickets.TicketGuid =
            System.Guid.NewGuid().ToString();

        // Save the new Help Desk Ticket
        var result =
            SyncfusionHelpDeskService
                .CreateTicketAsync(DbFactory, NewHelpDeskTickets);

        // Clear the form
        objHelpDeskTicket =
            new HelpDeskTicketDTO() { TicketDate = DateTime.Now };

        // Show the Toast
        ToastContent = "Saved!";
        await Task.Delay(100);
        await this.ToastObj.ShowAsync();
    }
    catch (Exception ex)
    {
        ToastContent = ex.GetBaseException().Message;
        await this.ToastObj.ShowAsync();
    }
}

```

Test the form

We can now run the project and enter new help desk tickets.

The screenshot shows a web application interface for 'SyncfusionHelpDesk'. On the left is a dark blue sidebar with navigation links: Home (with a house icon), Counter (with a plus icon), Weather (with a menu icon), Auth Required (with a lock icon), Register (with a person icon), and Login (with a person icon). The main content area is white and titled 'New Help Desk Ticket'. It contains a dropdown menu with 'New' selected, a 'Ticket Date' field with '12/1/2024' and a calendar icon, a 'Ticket Description' text area, and a 'Requester Email' text field. A blue 'Save' button is at the bottom left. An 'About' link is in the top right corner.

Figure 54: New help desk ticket form

The user is then presented with a new help desk ticket form.

This is a close-up of the dropdown menu from the 'New Help Desk Ticket' form. The dropdown is open, showing four options: 'New' (highlighted in blue), 'Open' (highlighted in light gray), 'Urgent', and 'Closed'. The dropdown has a green border and a small downward arrow on the right side of the header.

Figure 55: Select ticket status

They can select a ticket status by using the Dropdown List control.

The screenshot shows a form titled "Ticket Date". At the top, there is a text input field containing "12/1/2024" and a calendar icon. Below this is a date picker calendar for "December 2024". The calendar has a grid with days of the week (S, M, T, W, T, F, S) and dates (1 through 31). The date "1" is highlighted in blue. At the bottom of the calendar is a blue button labeled "Today".

Figure 56: Select ticket date

The user can select a ticket date by using the DatePicker control.

The screenshot shows a form titled "Ticket Description". It has two text input fields: "Ticket Description" and "Requester Email". Below the "Ticket Description" field is a red error message: "The Ticket Description field is required." Below the "Requester Email" field is a red error message: "The Ticket Requester Email field is required." At the bottom left is a blue button labeled "Save". A red box highlights the "Save" button, and a red arrow points to it from the word "Validation" in large red text. Another red arrow points from the word "Validation" to the "Requester Email" field. A third red arrow points from the "Ticket Description" field to the "Requester Email" field.

Figure 57: Validation errors

If the user tries to save an incomplete record, they will see validation errors like the ones shown in Figure 57.

New Help Desk Ticket

Ticket Date

Ticket Description

Requester Email

Figure 58: Save ticket

With a properly completed form, the user can click the Save button to save the data.

Help Desk Ticket

Saved!

Figure 59: Toast confirmation

They will see a brief confirmation message in the Toast control.

```

SELECT TOP (1000) [Id]
, [TicketStatus]
, [TicketDate]
, [TicketDescription]
, [TicketRequesterEmail]
, [TicketGUID]
FROM [SyncfusionHelpDesk].[dbo].[HelpDeskTickets]
  
```

Id	TicketStatus	TicketDate	TicketDescription	TicketRequesterEmail	TicketGUID
1	New	2024-12-01 00:00:00.000	My computer won't start	User@email.com	c61b012d-7611-4538-bdaf-5813d2cb6c19

Figure 60: View data in the database

If we look in the database, we will see the data has been added.

Enter data for at least six help desk tickets so that you will have enough data to demonstrate paging in the Syncfusion DataGrid, which is covered in the following chapter.

Chapter 8 Help Desk Ticket Administration

In this chapter, we will detail the steps to create screens that allow administrators to manage help desk tickets.

In the ticket administration page that we will construct, help desk tickets can be updated and deleted, but ticket details can only be added.

Create the ticket administration page

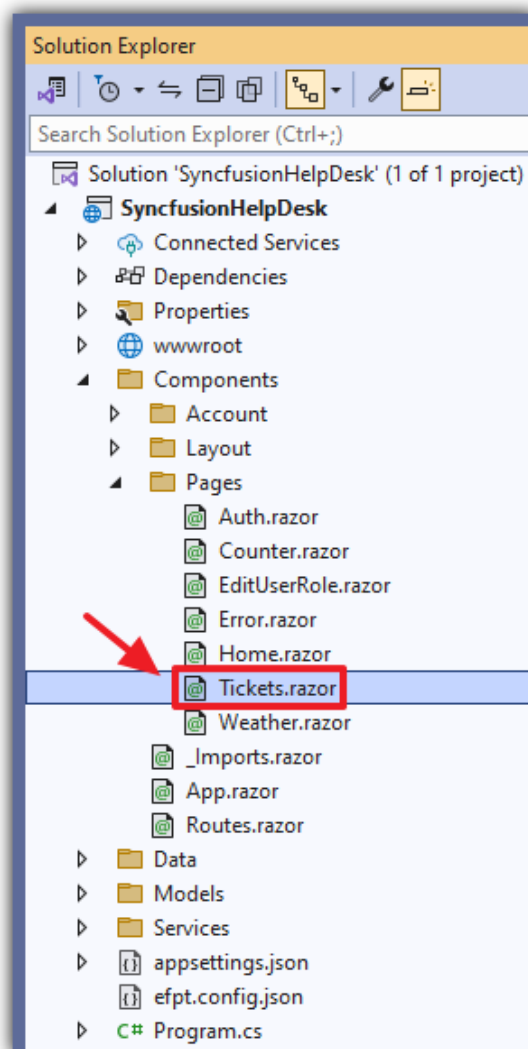


Figure 61: New item

We will now create a page that will allow all users to see their tickets, and for administrators to see all tickets.

1. Add the ticket administration page to the project and name it **Tickets.razor**.
2. Use the following code for the Razor control:

Code Listing 36: Tickets.razor

```
@page "/tickets"
@using Microsoft.AspNetCore.Identity
@using Microsoft.EntityFrameworkCore
@using SyncfusionHelpDesk.Data;
@using SyncfusionHelpDesk.Models
@using System.Security.Claims
@inject AuthenticationStateProvider AuthenticationStateProvider
@inject RoleManager<IdentityRole> RoleManager
@inject UserManager<ApplicationUser> UserManager
@inject IDbContextFactory<SyncfusionHelpDeskContext> DbFactory
@inject SyncfusionHelpDeskService SyncfusionHelpDeskService

@code {
    #nullable disable
    ClaimsPrincipal CurrentUser = new ClaimsPrincipal();

    private string PageTitle = "Administration";
    private string ADMINISTRATION_ROLE = "Administrators";
    private bool isAdmin = false;

    public IQueryable<HelpDeskTicket> colHelpDeskTickets { get; set; }
    private HelpDeskTicket SelectedTicket = new HelpDeskTicket();
    public bool EditDialogVisibility { get; set; } = false;
    public bool DeleteRecordConfirmVisibility { get; set; } = false;
}
```

Add link in NavMenu.razor

Here's how to add a link to the ticket control in the navigation menu:

1. Open the **NavMenu.razor** control in the **Shared** folder and remove the code for the links to the counter, auth, and weather pages from Code Listing 38.

Code Listing 37: Remove NavMenu Code

```
——<div class="nav-item px-3">
——<NavLink class="nav-link" href="counter">
——<span class="bi bi-plus-square-fill nav-menu" aria-
hidden="true"></span> Counter
——</NavLink>
```

```

</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" href="weather">
    <span class="bi bi-list-nested-nav-menu" aria-
hidden="true"></span> Weather
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" href="auth">
    <span class="bi bi-lock-nav-menu" aria-hidden="true"></span>
Auth Required
  </NavLink>
</div>

```

2. Add the code from Code Listing 39 under the **<Authorized>** tag.

Code Listing 38: Add NavMenu Code

```

<div class="nav-item px-3">
  <NavLink class="nav-link" href="tickets">
    <span class="bi bi-plus-square-fill-nav-menu" aria-
hidden="true"></span> Tickets
  </NavLink>
</div>

```

This will display a link to the Tickets.razor page, but this link will only be displayed for users who are logged in.

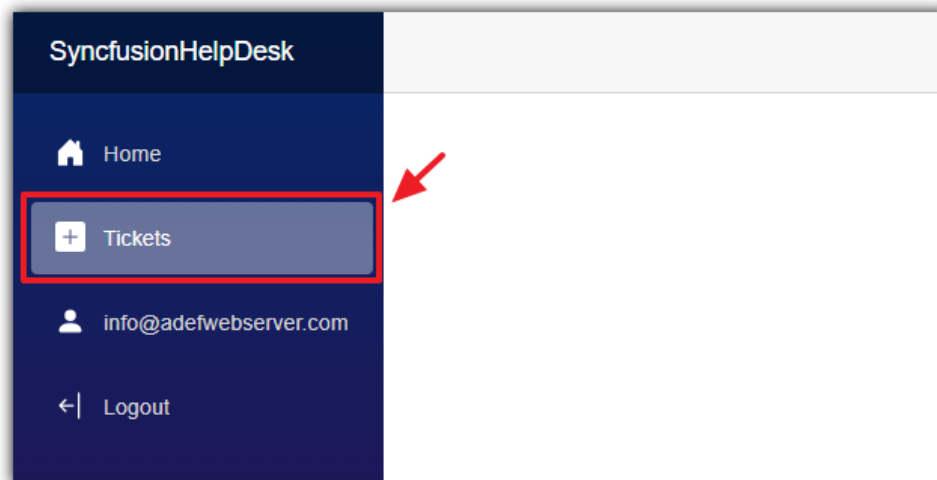


Figure 62: Tickets link

When we run the application and log in, we will see the link to the Tickets page. When we click on the link, we are brought to the Tickets page, which is currently blank.

Using the Syncfusion DataGrid

The Syncfusion DataGrid, programmatically called **SfGrid**, is a control that allows you to display tabular data. We will use it to display the help desk tickets. This control will allow us to page, sort, and trigger record editing.

It has a **DataSource** property that we will bind to a collection of the help desk tickets. It also has properties that allow for paging and sorting.

The control also allows us to define the columns, which will contain the tabular data, as well as Edit and Delete buttons. To use the Syncfusion DataGrid:

1. Add the following markup to the **Tickets.razor** file:

Code Listing 39: DataGrid

```
<style>
    .custom-dialog .e-dialog {
        max-height: 90vh !important; /* Customize the max height */
    }
</style>
<AuthorizeView>
    <Authorized>
        <h1>@PageTitle</h1>
        <div>
            <div id="target" style="height: 500px;">
                <SfGrid ID="Grid"
                    DataSource="@colHelpDeskTickets"
                    AllowPaging="true"
                    AllowSorting="true"
                    AllowResizing="true"
                    AllowReordering="true">
                    <GridPageSettings PageSize="5"></GridPageSettings>
                    <GridEvents CommandClicked="OnCommandClicked"
                        TValue="HelpDeskTicket">
                    </GridEvents>
                    <GridColumns>
                        <GridColumn HeaderText=""
                            TextAlign="TextAlign.Left" Width="150">
                            <GridCommandColumns>
                                <GridCommandColumn
                                    Type=CommandButtonType.Edit
                                    ButtonOption="@ (new
                                        CommandButtonOptions() {
                                            Content = "Edit",
                                            CssClass = "e-primary" })">
                                </GridCommandColumn>
                                @if (isAdmin)
                                {
```

```

                                <GridCommandColumn
Type=CommandButtonType.Delete
                                ButtonOption="@{(new
CommandButtonOptions() {
                                Content = "Delete",
                                CssClass = "e-danger"}})">
                                </GridCommandColumn>
                                }
                                </GridCommandColumns>
                                </GridColumn>
                                <GridColumn IsPrimaryKey="true"
Field=@nameof(HelpDeskTicket.Id)
                                HeaderText="ID #" TextAlign="@TextAlign.Left"
Width="70">
                                </GridColumn>
                                <GridColumn
Field=@nameof(HelpDeskTicket.TicketStatus)
                                HeaderText="Status" TextAlign="@TextAlign.Left"
Width="80">
                                </GridColumn>
                                <GridColumn
Field=@nameof(HelpDeskTicket.TicketDate)
                                HeaderText="Date" TextAlign="@TextAlign.Left"
                                Format="d" Width="80">
                                </GridColumn>
                                <GridColumn
Field=@nameof(HelpDeskTicket.TicketDescription)
                                HeaderText="Description"
TextAlign="@TextAlign.Left" Width="150">
                                </GridColumn>
                                <GridColumn
Field=@nameof(HelpDeskTicket.TicketRequesterEmail)
                                HeaderText="Requester"
TextAlign="@TextAlign.Left" Width="150">
                                </GridColumn>
                                </GridColumns>
                                </SfGrid>
                                </div>
                                </div>
                                </Authorized>
                                <NotAuthorized>
                                <p>You are not authorized to view this page.</p>
                                </NotAuthorized>
                                </AuthorizeView>

```

2. Add the following to the **@code** section:

Code Listing 40: DataGrid code

```
protected override async Task OnInitializedAsync()
{
    // Get the current user
    var authState = await AuthenticationStateProvider
        .GetAuthenticationStateAsync();

    CurrentUser = authState.User;

    // Check if the current user is an Administrator
    if (CurrentUser.Identity?.IsAuthenticated == true)
    {
        var CurrentApplicationUser =
            await UserManager
                .FindByNameAsync(CurrentUser.Identity.Name);

        // See if the current user is an Administrator
        isAdmin =
            await UserManager.IsInRoleAsync(
                CurrentApplicationUser, ADMINISTRATION_ROLE);
    }

    // GetHelpDeskTickets returns IQueryable that the
    // SfGrid will use to only pull records for the
    // page that is currently selected
    colHelpDeskTickets = SyncfusionHelpDeskService
        .GetHelpDeskTickets(
            DbFactory, isAdmin, CurrentUser.Identity.Name);

    // Set PageTitle
    PageTitle = isAdmin
        ? "Administration"
        : "My Help Desk Tickets";
}

public async void OnCommandClicked(
    CommandClickEventArgs<HelpDeskTicket> args)
{
    // Code to be added later
}
```

The **OnInitializedAsync** method is the first method to run when the page loads. This method populates the **colHelpDeskTickets** collection, which is bound to the **DataSource** property of the DataGrid.

Administration

	ID #	Status	Date	Description	Requester
Edit Delete	1	New	12/1/2024	My computer won't start	User@email.com
Edit Delete	2	Urgent	11/6/2024	My printer is out of ink	User@email.com
Edit Delete	3	New	11/20/2024	Forgot my password	Admin@Email.com
Edit Delete	4	New	11/29/2024	I need access to the paystub application	Admin@Email.com
Edit Delete	5	Open	12/1/2024	Can't log into the portal	user@adefwebserver.com

« < 1 2 > »

1 of 2 pages (6 items)

Figure 63: DataGrid control

Run the application, log in as an administrator, and navigate to the Tickets page. We will see the help desk ticket records displayed in the DataGrid.

	ID #	Status	Date	Description
Edit Delete	1	New	12/1/2024	My comp
Edit Delete	3	New	11/20/2024	Forgot m
Edit Delete	4	New	11/29/2024	I need ac
Edit Delete	1002	New	12/5/2024	Email ser
Edit Delete	5	Open	12/1/2024	Can't log

Figure 64: Sorting

Sorting is performed by clicking on the column headers.

« < 1 2 > »

1 of 2 pages (6 items)

Figure 65: Paging

The grid also enables paging.

Deleting a record

We will implement the functionality to delete a help desk ticket record. To enable this, the `DataGrid` allows us to create custom command buttons by using the `CommandClicked` property that we currently have wired up to the `OnCommandClicked` method.

We will use the `OnCommandClicked` method to display a pop-up window using the Syncfusion `Dialog` control, which will require the user to confirm that they want to delete the record.

Syncfusion Dialog control

The `Dialog` control is used to display information and accept user input. It can be displayed as a *modal control* that requires the user to interact with it before continuing to use any other part of the application. Here are the steps to implement it:

1. Enter the code from Code Listing 42 inside the `AuthorizeView` control tag.

Code Listing 41: Confirm delete dialog

```
<SfDialog Target="#target"
          Width="100px"
          Height="120px"
          IsModal="true"
          ShowCloseIcon="false"
          @bind-Visible="DeleteRecordConfirmVisibility">
  <DialogTemplates>
    <Header> DELETE RECORD ? </Header>
    <Content>
      <div class="button-container">
        <SfButton CssClass="e-primary"
          OnClick="ConfirmDeleteYes">Yes</SfButton>
        <SfButton OnClick="ConfirmDeleteNo">No</SfButton>
      </div>
    </Content>
  </DialogTemplates>
</SfDialog>
```

2. Change the `OnCommandClicked` method to the following to respond to the Delete button being selected for a help desk record. This method will open the confirmation dialog.

Code Listing 42: Open confirmation dialog

```
public async void OnCommandClicked(
    CommandClickEventArgs<HelpDeskTicket> args)
{
    // Get the selected Help Desk Ticket
    SelectedTicket =
        await SyncfusionHelpDeskService.GetHelpDeskTicketAsync(
            DbFactory, args.RowData.TicketGuid);
}
```



```

        if (args.CommandColumn.ButtonOption.Content == "Delete")
        {
            // Open Delete confirmation dialog
            this.DeleteRecordConfirmVisibility = true;
            StateHasChanged();
        }
    }
}

```



Note: *StateHasChanged*, which is used in Code Listing 43, notifies a component (in this case, the Tickets page) that its state has changed and causes that component to re-render. This is usually only required when that state was changed by a JavaScript interop call. The Syncfusion Dialog control uses JavaScript interop in the underlying code to open the dialog, so *StateHasChanged* is required in this case.

3. Add the method from Code Listing 44, which will just close the dialog if the user selects **No**.

Code Listing 43: Close dialog

```

public void ConfirmDeleteNo()
{
    // Open the dialog
    // to give the user a chance
    // to confirm they want to delete the record
    this.DeleteRecordConfirmVisibility = false;
}

```

Using @ref to capture references to components

Adding a **@ref** attribute to a component allows you to programmatically access and manipulate a control or component. To implement it:

1. Add the **@ref** attribute to a component, then define a field with the same type as the component.
2. Add the property from Code Listing 45 to the **SfGrid** control.

Code Listing 44: DataGrid @ref

```
@ref="gridObj"
```

3. Now, add the corresponding field to the **@code** section from Code Listing 46.

Code Listing 45: DataGrid property

```
SfGrid<HelpDeskTicket> gridObj;
```

4. Add the method from Code Listing 47 to delete the record when the user selects **Yes** in the dialog. This method will refresh the DataGrid by calling `gridObj.Refresh()`, which uses the `gridObj` object defined with the `@ref` attribute.

Code Listing 46: Delete record

```
public async void ConfirmDeleteYes()
{
    // The user selected Yes to delete the
    // selected Help Desk Ticket
    // Delete the record
    var result =
        await SyncfusionHelpDeskService.DeleteHelpDeskTicketsAsync(
            DbFactory, SelectedTicket);

    // Close the dialog
    this.DeleteRecordConfirmVisibility = false;

    // Refresh the SfGrid
    // so the deleted record will not show
    await gridObj.Refresh();
}
```

When we run the application, we can click the Delete button to open the dialog.

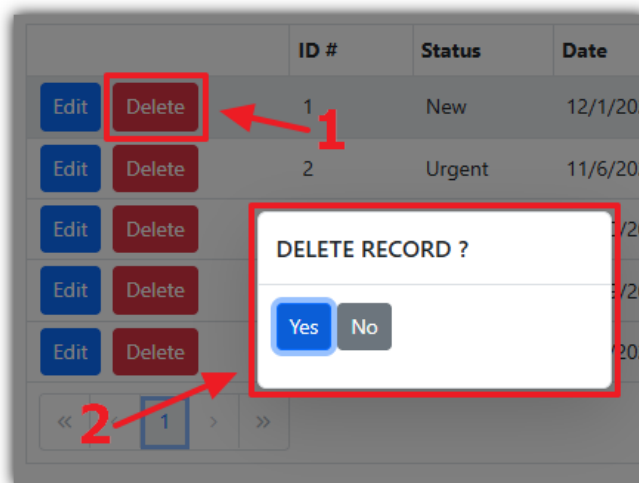


Figure 66: Delete confirmation

Selecting No will just close the dialog. Selecting Yes will delete the selected record and refresh the DataGrid.

Edit ticket control

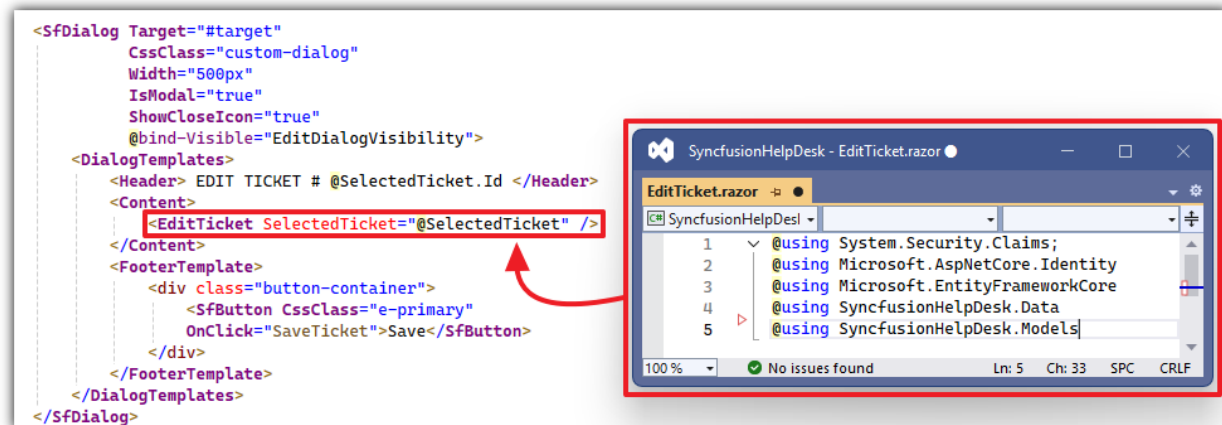


Figure 67: Edit ticket control

We will construct an EditTicket control that will be placed inside a dialog on the Tickets page, which will be displayed when you edit a help desk ticket. We do this to allow this control to be reused in the EmailTicketEdit.razor page covered in the following chapter. To construct the EditTicket control:

1. In the **Pages** folder, create a new control called **EditTicket.razor** using the code from Code Listing 48.

Code Listing 47: EditTicket.razor

```
@using System.Security.Claims;
@using Microsoft.AspNetCore.Identity
@using Microsoft.EntityFrameworkCore
@using SyncfusionHelpDesk.Data
@using SyncfusionHelpDesk.Models
@inject RoleManager<IdentityRole> RoleManager
@inject UserManager<ApplicationUser> UserManager
@inject AuthenticationStateProvider AuthenticationStateProvider
@inject IDbContextFactory<SyncfusionHelpDeskContext> DbFactory
@inject SyncfusionHelpDeskService SyncfusionHelpDeskService
```

2. Next, add the code from Code Listing 49 for the markup of the control.

Code Listing 48: EditTicket markup

```
<!-- This style will show -->
<!-- the entire content of TicketDescription -->
<!-- on hover over the field -->
<style>
    .e-grid .e-rowcell {
        white-space: nowrap;
        overflow: hidden;
    }
</style>
```

```

        text-overflow: ellipsis;
    }
    .e-grid .e-rowcell:hover {
        overflow: visible;
    }
</style>
<div>
    <SfDropDownList Enabled="isAdmin" TItem="HelpDeskStatus"
TValue="string"
                PopupHeight="230px"
                FloatLabelType="@FloatLabelType.Always"
                Placeholder="Ticket Status"
                @bind-Value="@SelectedTicket.TicketStatus"
                DataSource="@HelpDeskStatusData.Statuses">
        <DropDownListEvents TItem="HelpDeskStatus" TValue="string" />
        <DropDownListFieldSettings Text="Text" Value="ID" />
    </SfDropDownList>
</div>

<div>
    <SfDatePicker ID="TicketDate" Enabled="isAdmin"
                Placeholder="Ticket Date"
                FloatLabelType="@FloatLabelType.Always"
                @bind-Value="@SelectedTicket.TicketDate"
                Max="DateTime.Now"
                ShowClearButton="false">
    </SfDatePicker>
</div>

<div>
    <SfTextBox Enabled="isAdmin" Placeholder="Ticket Description"
                FloatLabelType="@FloatLabelType.Always"
                @bind-Value="@SelectedTicket.TicketDescription">
    </SfTextBox>
</div>

<div>
    <SfTextBox Enabled="isAdmin" Placeholder="Requester Email"
                FloatLabelType="@FloatLabelType.Always"
                @bind-Value="@SelectedTicket.TicketRequesterEmail">
    </SfTextBox>
</div>

<br />

<SfGrid @ref="GridRef" DataSource="@SelectedTicket.HelpDeskTicketDetails">
    <GridColumn>
        <GridColumn Field="TicketDetailDate"
                HeaderText="Date"

```

```

                Format="MM/dd/yyyy"
                TextAlign="TextAlign.Left"
                Width="150">
            </GridColumn>
            <GridColumn Field="TicketDescription"
                HeaderText="Description"
                TextAlign="TextAlign.Left"
                Width="300">
            </GridColumn>
        </GridColumn>
    </SfGrid>
<br />

<SfTextBox Placeholder="New Help Desk Ticket Detail"
    @bind-Value="@NewHelpDeskTicketDetail">
</SfTextBox>

<br />
<br />

<SfButton CssClass="e-small e-success"
    @onclick="AddHelpDeskTicketDetail">
    Add Detail
</SfButton>

<br />
<br />

```

3. Add the following code, which implements the code for the control:

Code Listing 49: EditTicket code

```

@code {
#nullable disable
    [Parameter]
    public HelpDeskTicket SelectedTicket { get; set; }

    private SfGrid<HelpDeskTicketDetail> GridRef;

    private string ADMINISTRATION_ROLE = "Administrators";
    private bool isAdmin = false;
    private string NewHelpDeskTicketDetail = "";

    protected override async Task OnInitializedAsync()
    {
        var authState =
            await AuthenticationStateProvider
                .GetAuthenticationStateAsync();
    }
}

```

```

var user = authState.User;

if (user.Identity?.IsAuthenticated == true)
{
    var CurrentUser =
        await UserManager.FindByNameAsync(user.Identity.Name);

    // If they are an Administrator
    // Enable editing
    isAdmin =
        await UserManager
            .IsInRoleAsync(CurrentUser, ADMINISTRATION_ROLE);
}
}

private void AddHelpDeskTicketDetail()
{
    // Create New HelpDeskTicketDetails record
    HelpDeskTicketDetail NewHelpDeskTicketDetails =
        new HelpDeskTicketDetail();

    NewHelpDeskTicketDetails.HelpDeskTicketId =
        SelectedTicket.Id;

    NewHelpDeskTicketDetails.TicketDetailDate =
        DateTime.Now;

    NewHelpDeskTicketDetails.TicketDescription =
        NewHelpDeskTicketDetail;

    // Add to collection
    SelectedTicket.HelpDeskTicketDetails
        .Add(NewHelpDeskTicketDetails);

    // Clear the Text Box
    NewHelpDeskTicketDetail = "";

    // Update the Grid
    GridRef.Refresh();
}
}

```

Notice that this exposes a **SelectedTicket** parameter (of type **HelpDeskTickets**) that will accept a reference of a help desk ticket record.

4. In the Tickets.razor control, add the markup from Code Listing 51 to display the EditTicket.razor control in a new Dialog control.

Code Listing 50: EditTicket Dialog

```
<SfDialog Target="#target"
    CssClass="custom-dialog"
    Width="500px"
    IsModal="true"
    ShowCloseIcon="true"
    @bind-Visible="EditDialogVisibility">
    <DialogTemplates>
        <Header> EDIT TICKET # @SelectedTicket.Id </Header>
        <Content>
            <EditTicket SelectedTicket="@SelectedTicket" />
        </Content>
        <FooterTemplate>
            <div class="button-container">
                <SfButton CssClass="e-primary"
                    OnClick="SaveTicket">Save</SfButton>
            </div>
        </FooterTemplate>
    </DialogTemplates>
</SfDialog>
```

Note that this instantiates the `EditTicket` control (the `EditTicket.razor` code page) and passes a reference to the currently selected help desk ticket record (`@SelectedTicket`) through the `SelectedTicket` property.

5. Add the following to the **@code** section to implement the functionality for the Save button in the dialog:

Code Listing 51: SaveTicket method

```
public async Task SaveTicket()
{
    // Update the selected Help Desk Ticket
    var result =
        await SyncfusionHelpDeskService.UpdateTicketAsync(
            DbFactory, SelectedTicket);

    // Close the Edit dialog
    this.EditDialogVisibility = false;

    // Refresh the SfGrid
    // so the changes to the selected
    // Help Desk Ticket are reflected
    await gridObj.Refresh();
}
```

6. Add the following code to the **OnCommandClicked** method under the existing `SelectedTicket` line, to open the dialog when the Edit button is clicked on a row in the `DataGrid`.

Code Listing 52: Open dialog

```
if (args.CommandColumn.ButtonOption.Content == "Edit")
{
    // Open the Edit dialog
    this.EditDialogVisibility = true;
    StateHasChanged();
}
```

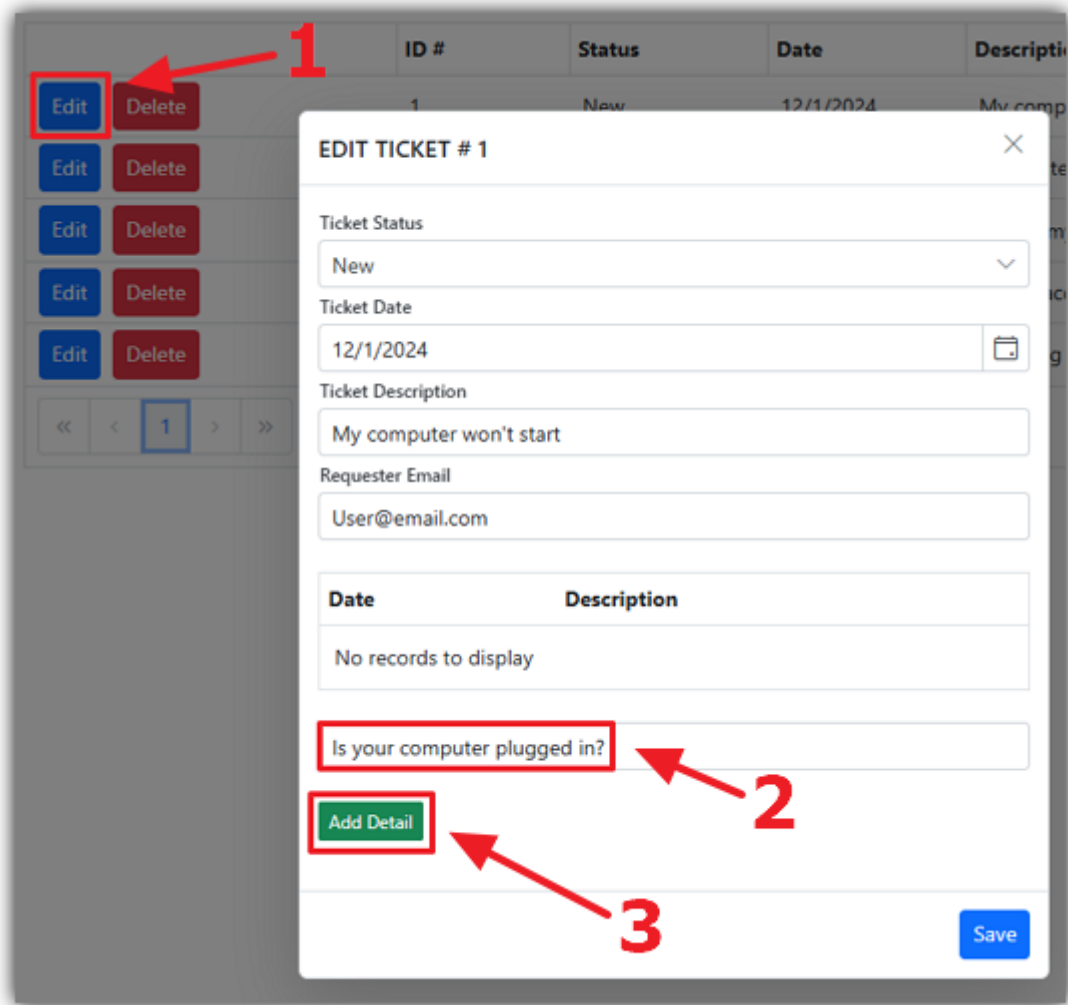


Figure 68: Edit ticket dialog

When we run the application, we can click the Edit button next to a record to open it in the dialog. The help desk ticket values at the top of the form can be edited and saved by clicking the Save button at the bottom of the dialog.

On the bottom half of the dialog, help desk ticket detail records can be added by entering text in the text box and clicking the Add Detail button.

EDIT TICKET # 1

×

Ticket Status

New

▼

Ticket Date

12/1/2024

📅

Ticket Description

My computer won't start

Requester Email

User@email.com

Date	Description
12/06/2024	Is your computer plugged in?

New Help Desk Ticket Detail

Add Detail

Save

Figure 69: Edit ticket

The help desk detail record will be added, but it will not be saved until the Save button is clicked.

Chapter 9 Sending Emails

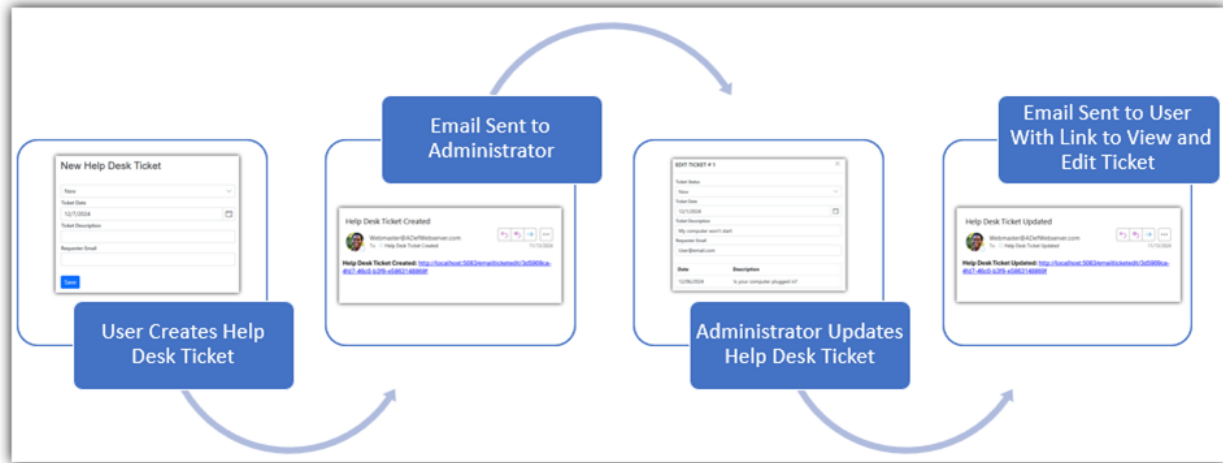


Figure 70: Email process

In this chapter, we will create code that will allow help desk ticket creators and administrators to update tickets by navigating to that help desk ticket record and clicking a link in an email.

We will also write code that will email the administrators when a new help desk ticket has been created, as well as code that will email help desk ticket creators and administrators when help desk tickets are updated.

Email using SendGrid

To enable emails, create a free SendGrid account at <https://sendgrid.com/> and obtain an email API key using the following steps:

1. Open the **appsettings.json** file and add the following two lines from Code Listing 54 below the opening curly bracket, entering your SendGrid key for the **SENDGRID_APIKEY** property and your email address for the **SenderEmail** property:

Code Listing 53: *appsettings.json*

```
"SENDGRID_APIKEY": "{ enter your key from app.sendgrid.com }",  
"SenderEmail": "{ enter your email address }",
```

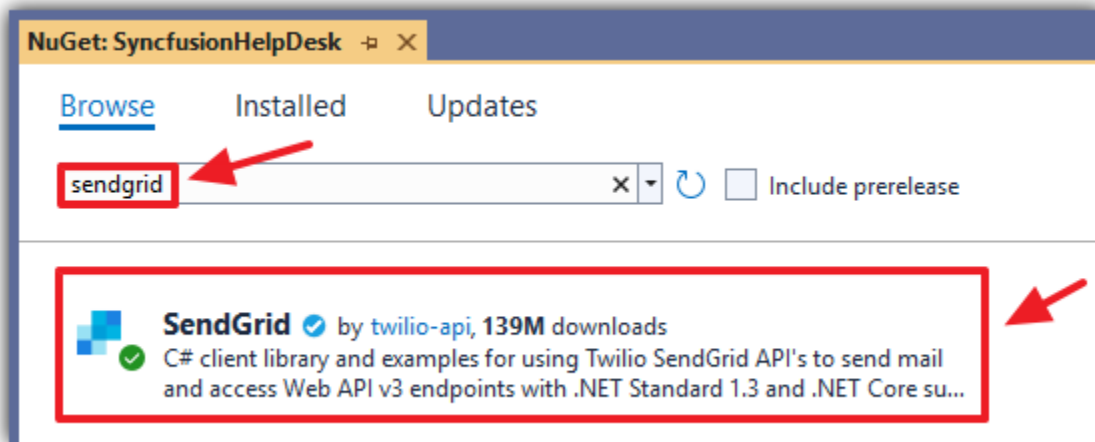


Figure 71: SendGrid NuGet package

2. Install the SendGrid NuGet package.

Email sender class

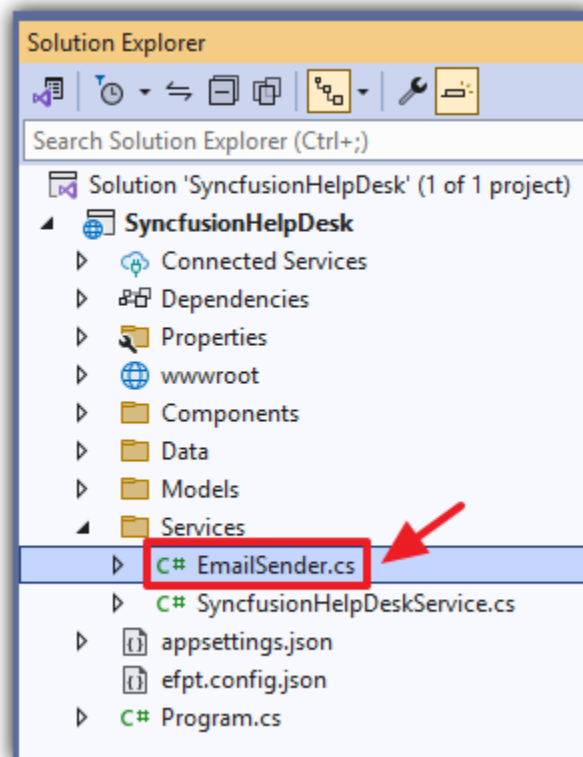


Figure 72: EmailSender.cs

Now, we will create a class that will read the settings from the appsettings.json file and send emails by using the following steps:

1. Create a new class in the **Services** folder called **EmailSender.cs** by using the code from Code Listing 55.

Code Listing 54: EmailSender.cs

```
using System;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Configuration;
using SendGrid;
using SendGrid.Helpers.Mail;

namespace SyncfusionHelpDesk
{
    public class EmailSender
    {
        private readonly IConfiguration configuration;
        private readonly IHttpContextAccessor httpContextAccessor;

        public EmailSender(
            IConfiguration Configuration,
            IHttpContextAccessor HttpContextAccessor)
        {
            configuration = Configuration;
            httpContextAccessor = HttpContextAccessor;
        }

        public async Task SendEmail(
            string EmailType,
            string EmailAddress,
            string TicketGuid)
        {
            try
            {
                // Email settings
                SendGridMessage msg = new SendGridMessage();
                var apiKey = configuration["SENDGRID_APIKEY"];
                var senderEmail = configuration["SenderEmail"];
                var client = new SendGridClient(apiKey);
                var FromEmail = new EmailAddress(
                    senderEmail,
                    senderEmail
                );

                // Format Email contents
                string strPlainTextContent =
                    $"{EmailType}: {GetHelpDeskTicketUrl(TicketGuid)}";
                string strHtmlContent =
                    $"<b>{EmailType}</b> ";
                strHtmlContent = strHtmlContent +
```

```

        $"<a href='{ GetHelpDeskTicketUrl(TicketGuid) }'>";
        strHtmlContent = strHtmlContent +
            $"{{GetHelpDeskTicketUrl(TicketGuid)}}</a>";

        if (EmailType == "Help Desk Ticket Created")
        {
            msg = new SendGridMessage()
            {
                From = FromEmail,
                Subject = EmailType,
                PlainTextContent = strPlainTextContent,
                HtmlContent = strHtmlContent
            };

            // Created Email always goes to Administrator
            // Send to senderEmail configured in appsettings.json
            msg.AddTo(new EmailAddress(senderEmail, EmailType));
        }

        if (EmailType == "Help Desk Ticket Updated")
        {
            msg = new SendGridMessage()
            {
                From = FromEmail,
                Subject = EmailType,
                PlainTextContent = strPlainTextContent,
                HtmlContent = strHtmlContent
            };

            // Updated emails go to Administrator or Ticket creator
            // Send to EmailAddress passed to method
            msg.AddTo(new EmailAddress(EmailAddress, EmailType));
        }

        var response = await client.SendEmailAsync(msg);
    }
    catch
    {
        // Could not send email
        // Perhaps SENDGRID_APIKEY not set in
        // appsettings.json
    }
}

// Utility

#region public string GetHelpDeskTicketUrl(string TicketGuid)
public string GetHelpDeskTicketUrl(string TicketGuid)
{

```

```

        var request = httpContextAccessor.HttpContext?.Request;

        var host = request?.Host.ToUriComponent();

        var pathBase = request?.PathBase.ToUriComponent();

        return
$@"{request?.Scheme}://{host}{pathBase}/emailticketedit/{TicketGuid}";
    }
    #endregion
}
}

```

2. To allow this class to be injected into our code as a service, add the following line to the **ConfigureServices** section of the **Program.cs** file:

Code Listing 55: EmailSender in Program.cs

```
builder.Services.AddScoped<EmailSender>();
```

Send emails: New help desk ticket

To send an email to the administrator when a new help desk ticket is created:

1. Open the **Home.razor** file and add the line from Code Listing 57 to inject the email service.

Code Listing 56: Inject EmailSender s3service

```
@inject EmailSender _EmailSender
```

2. Next, add the following code to the end of the **HandleValidSubmit** method.

Code Listing 57: HandleValidSubmit method

```

// Send Email
await _EmailSender.SendEmail(
    "Help Desk Ticket Created",
    "", // No need to pass an email because it goes to Administrator
    NewHelpDeskTickets.TicketGuid
);

```

Send emails: Updated help desk ticket

To send an email to the help desk ticket creator when the ticket is updated:

1. Open the **Tickets.razor** file and add the line from Code Listing 59 to inject the email service.

Code Listing 58: Add EmailSender service to tickets

```
@inject EmailSender _EmailSender
```

2. Next, add the following code to the end of the **SaveTicket** method:

Code Listing 59: Send email

```
// Send email to Requester
await _EmailSender.SendEmail(
    "Help Desk Ticket Updated",
    SelectedTicket.TicketRequesterEmail,
    SelectedTicket.TicketGuid
);
```

Route parameters

When a help desk ticket is initially created and saved to the database, it is assigned a unique GUID value. When an email is sent to notify the help desk ticket creator and administrator, the email will contain a link that passes this GUID to the Blazor control that we will create. This control will be decorated with a **@page** directive that contains a route parameter. To implement route parameters:

1. Create a new control called **EmailTicketEdit.razor** with the following code:

Code Listing 60: EmailTicketEdit route

```
@page "/emailticketedit/{TicketGuid}"
```

This line, together with a field in the **@code** section called **TicketGuid** (of type **string**), will allow this control to be loaded and passed a value for TicketGuid from a link in the email.

2. Enter the following code as the markup code for the file:

Code Listing 61: EmailTicketEdit markup code

```
@using Microsoft.EntityFrameworkCore
@using Microsoft.Extensions.Configuration
@using System.Security.Claims;
@using SyncfusionHelpDesk.Data;
@using SyncfusionHelpDesk.Models
@inject EmailSender _EmailSender
@inject IConfiguration _configuration
@inject AuthenticationStateProvider AuthenticationStateProvider
@inject IDbContextFactory<SyncfusionHelpDeskContext> DbFactory
@inject SyncfusionHelpDeskService SyncfusionHelpDeskService
```

```

<style>
    .custom-dialog .e-dialog {
        max-height: 90vh !important; /* Customize the max height */
    }
</style>
<div id="target" style="height: 500px;">
    @if (!EditDialogVisibility)
    {
        <h2>Your response has been saved</h2>
        <h2>Thank You!</h2>
    }
</div>
<SfDialog Target="#target"
    CssClass="custom-dialog"
    Width="500px"
    IsModal="true"
    ShowCloseIcon="true"
    Visible="EditDialogVisibility">
    <DialogTemplates>
        <Header> EDIT TICKET # @SelectedTicket.Id</Header>
        <Content>
            <EditTicket SelectedTicket="@SelectedTicket" />
        </Content>
        <FooterTemplate>
            <div class="button-container">
                <SfButton CssClass="e-primary"
                    OnClick="SaveTicket">Save</SfButton>
            </div>
        </FooterTemplate>
    </DialogTemplates>
</SfDialog>

```

3. Finally, enter the following code for the file:

Code Listing 62: EmailTicketEdit code

```

@code {
#nullable disable
    [Parameter] public string TicketGuid { get; set; }

    ClaimsPrincipal CurrentUser = new ClaimsPrincipal();
    private HelpDeskTicket SelectedTicket = new HelpDeskTicket();
    private bool EditDialogVisibility = true;

    private string ADMINISTRATION_ROLE = "Administrators";

    protected override async Task OnInitializedAsync()
    {
        // Get current user
    }
}

```



```

        var authState =
            await AuthenticationStateProvider
                .GetAuthenticationStateAsync();

        CurrentUser = authState.User;
    }

    protected override async Task
        OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            // Get the Help Desk Ticket associated with
            // the GUID that was passed to the control
            SelectedTicket =
                await SyncfusionHelpDeskService
                    .GetHelpDeskTicketAsync(DbFactory, TicketGuid);

            StateHasChanged();
        }
    }

    public async Task SaveTicket()
    {
        // Save the Help Desk Ticket
        var result =
            await SyncfusionHelpDeskService
                .UpdateTicketAsync(DbFactory, SelectedTicket);

        // Close the Dialog
        EditDialogVisibility = false;

        // Send Emails
        if (CurrentUser.Identity.IsAuthenticated)
        {
            if (CurrentUser.IsInRole(ADMINISTRATION_ROLE))
            {
                // User an Administrator
                // Send email to Requester
                await _EmailSender.SendEmail(
                    "Help Desk Ticket Updated",
                    SelectedTicket.TicketRequesterEmail,
                    SelectedTicket.TicketGuid
                );

                return;
            }
        }
    }
}

```

```

// User is not an Administrator
// Send email to Administrator
string AdministratorEmail =
_configuration["SenderEmail"];

await _EmailSender.SendEmail(
    "Help Desk Ticket Updated",
    AdministratorEmail,
    SelectedTicket.TicketGuid
);
    }
}

```

Notice that this page also includes the EditTicket control, effectively reusing that control for both this page and the tickets page.

Email link

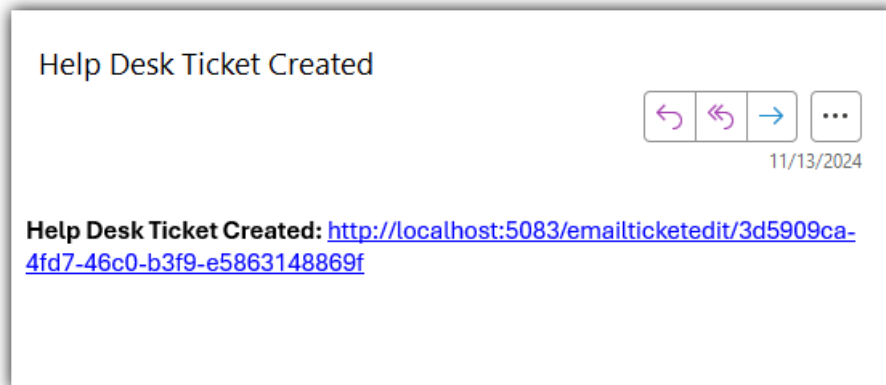


Figure 73: Ticket created email link

When we run the application and create a new help desk ticket, the administrator is sent an email with a link. Clicking that link will take the administrator directly to the help desk ticket.

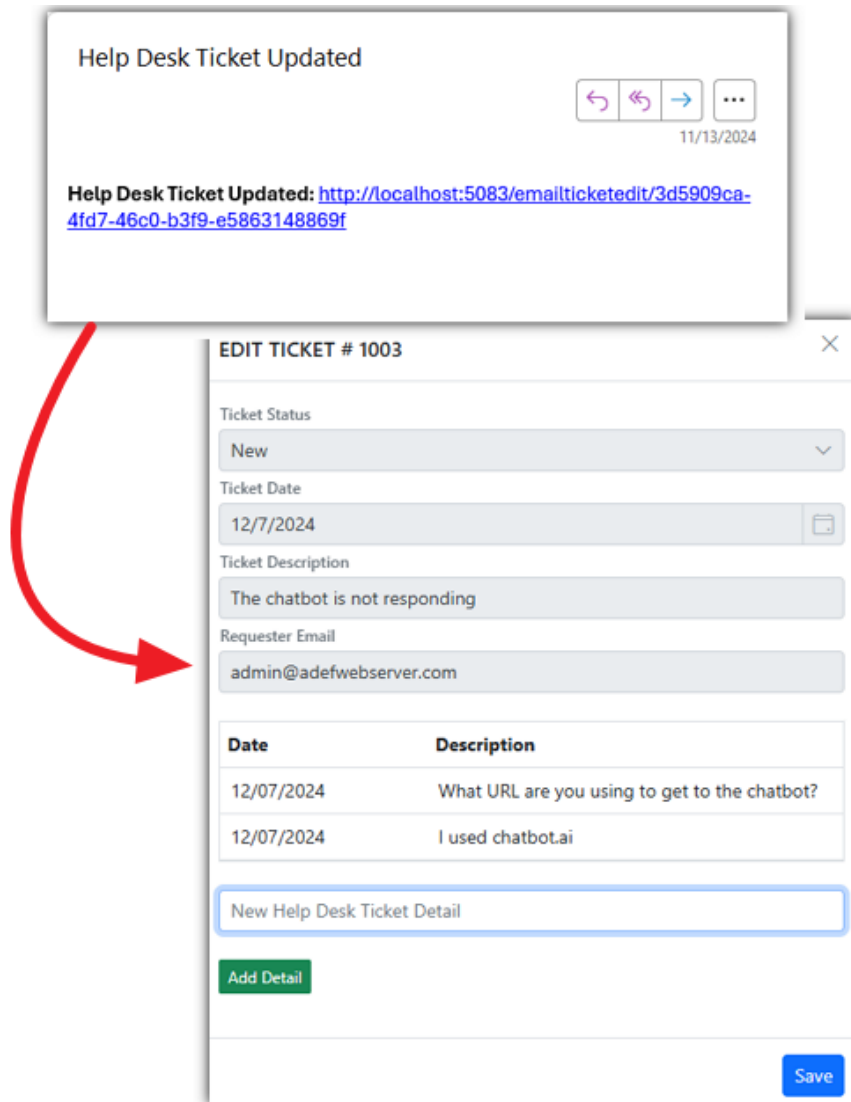


Figure 74: Ticket updated email link

When the administrator responds to the ticket, the user is sent an email with a link to the ticket that allows them to view and update the ticket.

Chapter 10 Creating Tickets Using AI

AI-Enhanced Help Desk Ticket Creation

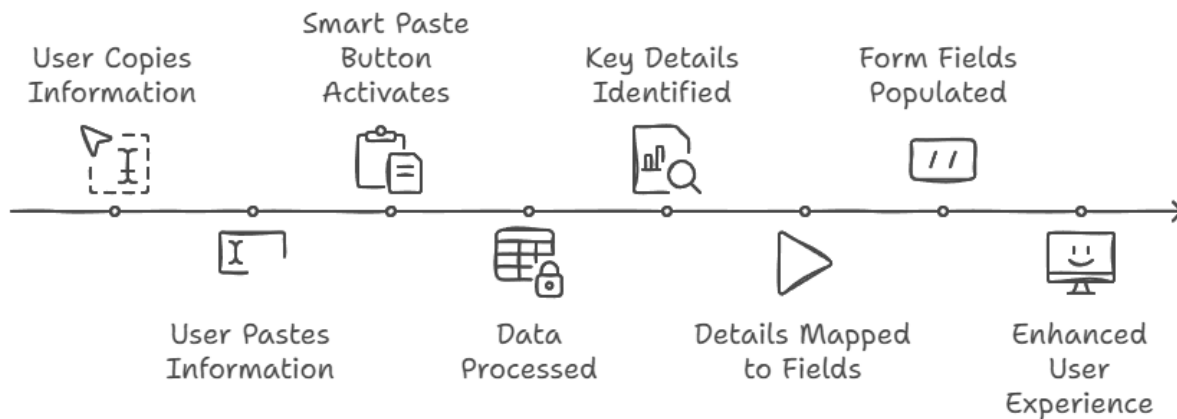


Figure 75: New ticket process

Traditionally, creating a help desk ticket requires users to manually fill out all form fields, which can be time-consuming and can leave you prone to error. By integrating AI capabilities, we can streamline this process by allowing users to copy relevant information from another source, such as an email, and use AI to intelligently populate the form fields.

This is achieved by using the Syncfusion Smart Paste Button component, which processes the pasted data, identifies key details like description and requester, and maps them to the corresponding form fields. This not only saves time but also ensures accuracy and consistency, enhancing the overall user experience.

Set up the Smart Paste Button component

We will now set up the Syncfusion Smart Paste Button component.

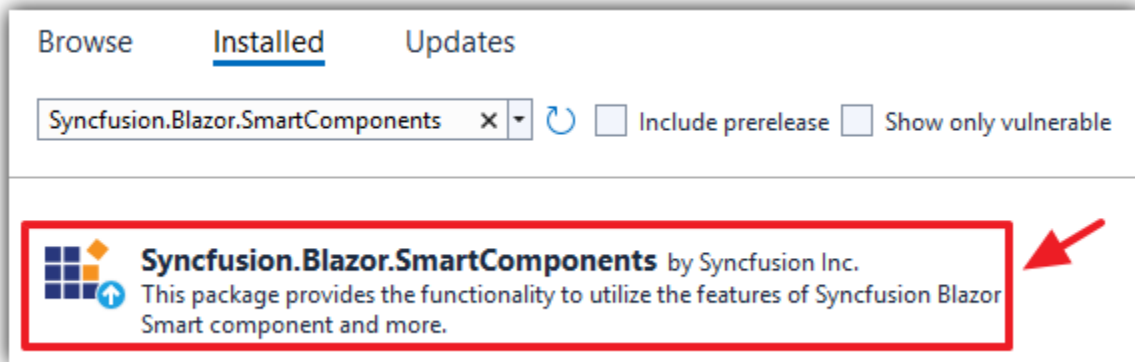


Figure 76: SmartComponents NuGet package

The Smart Paste Button component is contained in the Syncfusion.Blazor.SmartComponents NuGet package that we previously installed. To implement the Smart Paste Button:

1. Obtain a key to connect to the OpenAI service. You can obtain it by creating an account with one of the following services. Use the following links for instructions on obtaining a key for each of the services:
 - **OpenAI (recommended):** <https://help.openai.com/en/articles/4936850-where-do-i-find-my-openai-api-key>
 - **Azure OpenAI:** <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/create-resource?pivots=web-portal>
2. Open the **appsettings.json** file and add the following below the opening curly bracket, entering your OpenAI key for the **OpenAI:apiKey** property and the model you want to use for the **OpenAI:deploymentName** property (for example, *gpt-4o*). Optionally, you can add the **OpenAI:endpoint** property if you are using Azure OpenAI.

Code Listing 63: OpenAI appsettings.json

```
"OpenAI": {
  "apiKey": "{{ Your OpenAI Key }}",
  "deploymentName": "gpt-4o",
  "endpoint": ""
},
```



Note: For more details on setting up and using this component, see *Getting Started with Smart Paste Button Component* at: <https://blazor.syncfusion.com/documentation/smart-paste/getting-started>

Next, to programmatically read the values and pass them to the Syncfusion smart component, open the **Program.cs** file and add the following code before the **var app = builder.Build();** line.

Code Listing 64: OpenAI key in Program.cs

```
var apiKey = builder.Configuration["OpenAI:apiKey"];
```

```

var deploymentName = builder.Configuration["OpenAI:deploymentName"];
var endpoint = builder.Configuration["OpenAI:endpoint"];

if (apiKey != "")
{
    builder.Services.AddSyncfusionSmartComponents()
        .ConfigureCredentials(new
            AIServiceCredentials(apiKey, deploymentName, endpoint))
        .InjectOpenAIInference();
}

```

In the code of the **Home.razor** page, add the following markup after the markup for the existing **SfButton**:

Code Listing 65: SfSmartPasteButton markup

```

<SfSmartPasteButton
    IsPrimary="true"
    Content="Smart Paste"
    IconCss="e-icons e-paste">
</SfSmartPasteButton>

```

Using the Smart Paste Button

To use the Smart Paste Button:

1. Run the application and navigate to the new help desk ticket page. You can copy a message such as this:

Hello, my name is Michael, and I need Visual Studio installed. You can reach me at Webmaster@BlazorHelpWebsite.com. Thank you.

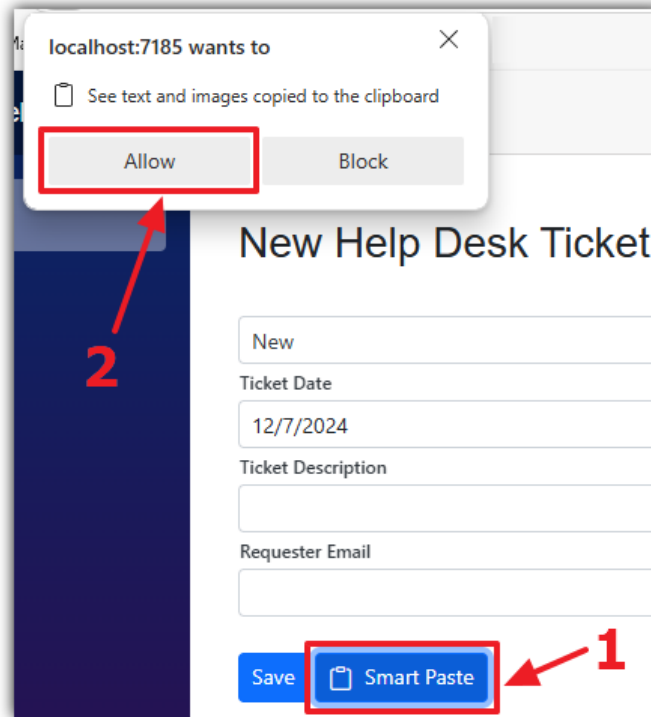


Figure 77: Allow access to the clipboard

2. When you select **Smart Paste** and it is the first time you have used it, you will see a pop-up window asking for permission to access your clipboard. Select **Allow**.

A screenshot of the "New Help Desk Ticket" form, now filled out. The "Ticket Description" field contains the text "Visual Studio installation request" and the "Requester Email" field contains "Webmaster@BlazorHelpWebsite.com". Red arrows point to these two fields. The "Save" and "Smart Paste" buttons are still visible at the bottom.

Figure 78: Ticket form filled by AI

The copied text will be parsed, and the help desk ticket form will be populated.

Conclusion

Now that we've seen how Blazor technology enables you to create sophisticated, manageable, and extensible applications using C# and Razor syntax, try it yourself!